M06 — Prácticas integrales: comunicación entre microservicios con .NET 8

Autor del curso: JAIME GALLO

Cómo utilizar el manual

Continúa exactamente `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria`: 22 proyectos, 208 pruebas y 28 documentos Markdown en `docs`. Construye cada una de las tres etapas en una carpeta nueva copiando la etapa anterior. Sigue los 72 pasos y aplica los bloques en el orden indicado.

Requisitos: Visual Studio con desarrollo ASP.NET, SDK .NET 8 y SQL Server Express o LocalDB. Conserva `global.json`, los proyectos hexagonales y el dominio de Producción. Los catálogos de este módulo son ejercicios de análisis: sus nombres e indicadores no demuestran tráfico HTTP, publicación, entrega ni consumo real. La implementación funcional distribuida en los pasos 5 a 15 de cada punto proporciona la comprobación separada de ese comportamiento.

SQL Server para las pruebas. Conserva `tests/Compartido/SqlServerPruebas.cs`. El helper lee `ACERIA_TEST_SQLSERVER`, conecta a `master` y crea una base con nombre único para cada prueba; la cuenta necesita permiso para crearla y eliminarla. Ejemplo PowerShell con LocalDB: `$env:ACERIA_TEST_SQLSERVER = "Server=(localdb)\MSSQLLocalDB;Database=master;Integrated Security=True;TrustServerCertificate=True"`. Para Express sustituye el servidor por `.\SQLEXPRESS`. Si no defines la variable, el helper utiliza Docker y `Testcontainers.MsSql` 3.10.0 con SQL Server 2022 Express y la imagen fija `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`. Una conexión inválida hace fallar las pruebas; no se omiten ni se sustituyen por un proveedor en memoria.

Convención de código. Cada bloque identifica archivo y operación. Crear exige un archivo nuevo; reemplazar exige un archivo existente; insertar exige una única coincidencia del ancla. Las rutas son relativas a la raíz de la etapa. Se anotan solo decisiones y comprobaciones significativas, con referencia a su línea física; no se explican repetidamente las líneas vacías, los delimitadores o el código convencional. Los tres retos se aplican en copias optativas independientes y añaden una prueba cada uno; no se acumulan en las etapas obligatorias.

Resultados de comprobación

Punto	Tests nuevos de catálogo	Tests nuevos funcionales	Tests de la etapa	Proyectos	Documentos
Base M05 5.5	—	—	208	22	28
6.1	13	11	232	27	30
6.2	13	19	264	28	32
6.3	12	7	283	29	34

Preparar Docker Desktop en Windows para consumir RabbitMQ

El alumno trabaja en **Windows con PowerShell, Visual Studio, .NET 8 y SQL Server Express LocalDB**. En este módulo Docker Desktop solo aloja **RabbitMQ**: Producción, Inventario y (en M08) YARP se ejecutan directamente con `dotnet run`. El despliegue conjunto de todas las aplicaciones en contenedores queda fuera de esta práctica.

1. Instala y abre Docker Desktop en Windows con **WSL 2** y contenedores **Linux**. Antes de arrancar el broker ejecuta `docker version`, `docker compose version` y `docker info --format '{{.OSType}}'`; la última comprobación debe mostrar `linux`. Si Docker no responde, inicia Docker Desktop y repite la comprobación.
2. Abre PowerShell en la **carpeta de la etapa concreta** que estés trabajando, donde debe existir `compose.yml` (se crea en M06 y se hereda en M07/M08). Comprueba `Test-Path .\compose.yml`; debe devolver `True`. Ejecuta `docker compose -f .\compose.yml config --quiet`; cualquier error indica que no puedes continuar.
3. Ejecuta `docker compose -f .\compose.yml up -d rabbitmq` y comprueba el estado con `docker compose -f .\compose.yml ps`. Espera al arranque real con `docker compose -f .\compose.yml exec -T rabbitmq rabbitmq-diagnostics -q ping`; si devuelve error, espera unos segundos y repite, sin dar la práctica por superada.
4. Comprueba los puertos locales `5672` (AMQP) y `15672` (administración): `Test-NetConnection localhost -Port 5672` y `Test-NetConnection localhost -Port 15672` deben devolver `TcpTestSucceeded=True`. Abre `http://localhost:15672/` con `curso` / `curso-local` **solo en este laboratorio**.
5. En **cada terminal PowerShell que ejecute un servicio**, configura `$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'` y, cuando el ejercicio indique mensajería, `$env:Comunicacion__WorkersActivos = 'true'`. Los procesos no heredan las variables de otras ventanas. Conserva LocalDB como motor SQL, salvo que el paso exija explícitamente otra instalación.
6. Si ya tienes otro broker en `5672`, no levantes dos Compose con los mismos puertos: utiliza el que está activo si coincide con la configuración del ejercicio o detén primero el anterior. Para detener sin borrar mensajes: `docker compose -f .\compose.yml stop rabbitmq`; para recuperar: `docker compose -f .\compose.yml start rabbitmq` y repite el diagnóstico. No uses `down -v` cuando debas conservar el volumen y las proyecciones.
7. Si aparecen errores, consulta `docker compose -f .\compose.yml logs --tail 80 rabbitmq`, revisa los puertos ocupados y la salud del broker. Un contenedor arrancado no demuestra que el flujo de outbox/inbox funcione: ejecuta después las peticiones, comprobaciones y pruebas E2E del punto.

Criterio para continuar: broker disponible, conexión AMQP verificada y servicios Windows listos para consumirlo. La validación definitiva de mensajería se realiza en los procedimientos funcionales del módulo, no solo mediante `docker compose ps`.

Entorno para las comunicaciones reales

Trabaja en Windows con .NET 8 y SQL Server Express LocalDB. Para las pruebas de integración, `ACERIA_TEST_SQLSERVER` debe apuntar a `master` con permisos de crear y eliminar bases aisladas. Si no se define, el helper heredado inicia SQL Server Express con Testcontainers y necesita Docker Desktop en modo contenedores Linux. Ningún test cambia silenciosamente a SQLite o InMemory.

```
$env:ACERIA_TEST_SQLSERVER='Server=(localdb)
\MSSQLLocalDB;Database=master;Trusted_Connection=True;TrustServerCertificate=True'
```

Desde 6.2, RabbitMQ también es necesario. El archivo `compose.yml` creado en 6.1 usa una imagen fijada y puertos locales. Desde la carpeta de la etapa, ejecuta:

```
docker compose up -d rabbitmq
$env:ACERIA_TEST_RABBITMQ='amqp://curso:curso-local@localhost:5672/'
```

La consola de administración queda en `http://localhost:15672/`, con usuario `curso` y clave de laboratorio `curso-local`. Abrirla muestra conexiones, colas y mensajes, pero no sustituye las pruebas automáticas. El volumen conserva mensajes entre reinicios. `docker compose stop` detiene el broker sin borrar el volumen; no ejecutes `down -v` si necesitas conservar mensajes pendientes.

Cada etapa tiene nombres distintos de base SQL y de topología RabbitMQ. `EnsureCreated` se utiliza aquí sobre bases didácticas nuevas: no migra una base que ya existía con otro esquema. Conserva las bases de etapas anteriores; no ejecutes comandos de borrado sobre ellas.

Arranque de los dos procesos

Al terminar el código de cada punto, abre dos terminales en su carpeta. En la primera inicia Inventario:

```
$env:ASPNETCORE_ENVIRONMENT='Development'
$env:Comunicacion__WorkersActivos='false'
dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile --urls http://localhost:5106
```

En la segunda inicia Producción, que conserva las API anteriores:

```
$env:ASPNETCORE_ENVIRONMENT='Development'
$env:Comunicacion__WorkersActivos='false'
dotnet run --project src/AceriaApp.Api --no-launch-profile --urls http://localhost:5105
```

En 6.1, deja `WorkersActivos=false`: solo existe el recorrido HTTP. Desde 6.2, cambia esa variable a `true` en ambas terminales antes de iniciar los procesos, con RabbitMQ disponible. El valor no se incluye como activo por defecto para que las pruebas heredadas de M5 puedan seguir alojando la API sin iniciar trabajo externo no relacionado con su caso.

En una tercera terminal, consulta `http://localhost:5105/api/v1/comunicacion/materiales/Acero`: la petición sale por HTTP hacia Inventario y devuelve el stock inicial 300. Un material inexistente devuelve404. Detén Inventario y repite: Producción devuelve503, no404 ni un stock inventado. El endpoint es informativo y no descuenta existencias.

Punto 6.1 — Comunicación síncrona mediante HTTP

Comprender una consulta HTTP entre dos procesos independientes

Producción necesita conocer la disponibilidad de un material, e Inventario ofrece esa información mediante HTTP. La llamada síncrona permite obtener una respuesta durante la petición original, pero también introduce dependencia temporal: si Inventario no responde, Producción tiene que decidir cómo comunicar ese fallo. Estudiaremos el contrato, el cliente HTTP y el comportamiento ante errores como una misma operación, no como configuraciones independientes.

Partimos de la aplicación de M05, que ya incluye el dominio de Producción, y trabajamos sobre una copia para conservar su estado. Los catálogos de llamadas son material de análisis; tener una propiedad booleana que indique reintentos no activa automáticamente una política de resiliencia. La comprobación significativa consiste en ejecutar las dos API en Windows y observar una consulta real de Producción a Inventario.

Carpeta de trabajo: `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar por separado el análisis y la comunicación ejecutable. La etapa termina con 232 pruebas en 27 proyectos.

Ejercicio guiado. Incorpora `LlamadaSincrona`, `ValidadorComunicacionSincrona` y `CatalogoLlamadas`, dos documentos nuevos y 13 pruebas de catálogo. Los bloques de integración de los pasos 5 a 15 implementan el comportamiento remoto.

PASO 6.1.1 — Crear la estructura del módulo 6 y del punto 6.1

Al copiar la etapa final de M05 conservamos los contratos y las pruebas sobre los que se construye la comunicación. La copia permite contrastar cambios sin alterar el punto de partida ni arrastrar archivos temporales de compilación.

1. Localiza la etapa completa `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http` y copia en ella el contenido de `5.5-dominio-aceria`; en Windows usa `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `6.1-comunicacion-sincrona-http`.
4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `5.5-dominio-aceria` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `5.5-dominio-aceria`: verifica la ruta antes de editar.

PASO 6.1.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `6.1-comunicacion-sincrona-http` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 22 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 22 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 6.1.3 — Verificar que la carpeta `docs` contiene los documentos del punto 5.5

El punto conserva 28 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 28 archivos Markdown.
2. Localiza los ADR-010 a ADR-023 y las fichas temáticas heredadas.

3. Compara nombres y contenido con `docs` de `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria`.
4. Comprueba que aún no existen `docs/comunicacion-sincrona-http.md` ni `docs/ADR-024-comunicacion-sincrona-http.md`.

Verificación concreta. Hay 28 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 6.1.4 — Crear el documento de comunicación síncrona en `docs`

La ficha separa el análisis de los contratos y la evidencia ejecutable.

1. Crea `docs/comunicacion-sincrona-http.md` dentro de la copia de `6.1-comunicacion-sincrona-http`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/comunicacion-sincrona-http.md` existe y que los 28 documentos anteriores permanecen intactos.

Archivo: `docs/comunicacion-sincrona-http.md`. **Operación:** Crear.

```
# Comunicación síncrona mediante HTTP — Punto 6.1

## Definición

Una interacción síncrona requiere recibir el resultado del destino para completar el paso del proceso.
En .NET, `await` permite esperar sin bloquear un hilo; no elimina la dependencia temporal entre servicios.

## HTTP

HTTP expresa métodos, recursos, cabeceras, cuerpos y estados. No toda respuesta no exitosa significa
ausencia: 404, 400 y 503 requieren tratamientos distintos. HTTP/1.1 y HTTP/2 utilizan TCP; HTTP/3 utiliza
QUIC sobre UDP.

## Clientes tipados

Una clase encapsula la dirección, serialización y traducción de respuestas del servicio remoto.
`IHttpClientFactory` gestiona los manejadores reutilizados; el cliente se registra con `AddHttpClient`.

## Configuración

Define URL base, timeout, cancelación y contratos de datos. Obtén las direcciones de la configuración del
entorno y escapa los segmentos que proceden del usuario. Un cliente tipado no debe ocultar errores de
transporte como un resultado de negocio inexistente.

## Acoplamiento temporal

La disponibilidad y latencia del destino forman parte del comportamiento observado por el solicitante. La
asincronía de una tarea .NET no convierte HTTP en mensajería.

## Riesgos

Latencia acumulada, saturación, respuestas tardías, indisponibilidad y fallos en cascada. Incluso tras un
timeout, el destino podría haber completado una escritura.

## Mitigación

Configura timeout, cancelación y límites de concurrencia. Usa reintentos acotados solo cuando la operación y
el contrato permitan repetirla. No reintentes POST o PATCH a ciegas: requiere una clave de idempotencia
aplicada de forma durable por el servidor. Un indicador del catálogo no implementa esa garantía. Un circuit
breaker limita llamadas durante una incidencia; no repara el servicio remoto. Un valor alternativo solo sirve
cuando el negocio lo acepta explícitamente.

## Consistencia
```

```
HTTP no proporciona una transacción atómica entre bases de servicios. Consultar stock no lo reserva. Una
operación distribuida necesita una política explícita de reserva, compensación o reconciliación.

## Cuándo usarlo

Favorece HTTP cuando el flujo necesita el resultado actual, existe un contrato de latencia y el acoplamiento
es aceptable. Una lectura también puede usar una proyección asíncrona si admite datos retrasados; la elección
se toma por interacción.

## Aplicación al curso

`LlamadaSincrona`, `CatalogoLlamadas` y `ValidadorComunicacionSincrona` describen tres consultas GET entre
nombres de servicios. Sus pruebas comprueban datos del análisis: no abren sockets ni prueban `HttpClient`,
reintentos o circuit breakers. La implementación funcional separada comprueba tráfico HTTP y sus fallos
controlados.
```

Verificación concreta. `docs/comunicacion-sincrona-http.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/comunicacion-sincrona-http.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 6.1.5 — Crear la carpeta `ComunicacionSincrona` en el proyecto API

La carpeta de comunicación síncrona agrupa los modelos con los que describimos llamadas y políticas. Su presencia no crea un cliente HTTP; ese comportamiento se incorporará mediante contratos, servicios y rutas concretas.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `ComunicacionSincrona` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/ComunicacionSincrona` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `5.5-dominio-aceria`.

Verificación concreta. Existe `src/AceriaApp.Api/ComunicacionSincrona` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

Implementación funcional: `compose.yml`

Compose levanta únicamente el broker del laboratorio. LocalDB sigue atendiendo a las aplicaciones en Windows. El volumen conserva datos del broker entre reinicios; estas credenciales son de práctica local y los puertos solo se publican en loopback.

Archivo completo: `compose.yml`. Crea el archivo y sus carpetas si no existen.

```
services:
  rabbitmq:
    image: rabbitmq:4.1.5-management
    environment:
      RABBITMQ_DEFAULT_USER: curso
      RABBITMQ_DEFAULT_PASS: curso-local
    ports:
      - "127.0.0.1:5672:5672"
      - "127.0.0.1:15672:15672"
    volumes:
```

```
      - rabbitmq-datos:/var/lib/rabbitmq
volumes:
  rabbitmq-datos:
```

Define la forma pública o la configuración reproducible que usarán los archivos del mismo punto.

Implementación funcional: `AceriaApp.Comunicacion.Aplicacion.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Aplicacion/AceriaApp.Comunicacion.Aplicacion.csproj`. Crea el archivo y sus carpetas si no existen.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Comunicacion.Contratos/AceriaApp.Comunicacion.Contratos.csproj" /
>
</ItemGroup>
</Project>
```

Declara una dependencia de compilación explícita; el ensamblado debe existir en la solución.

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add src\AceriaApp.Comunicacion.Aplicacion\AceriaApp.Comunicacion.Aplicacion.csproj
```

Implementación funcional: `IClienteInventario.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Aplicacion/IClienteInventario.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Contratos;
namespace AceriaApp.Comunicacion.Aplicacion;
public interface IClienteInventario
{
    Task<MaterialDto?> ObtenerAsync(string material, CancellationToken ct);
}
public sealed class InventarioNoDisponibleException(string mensaje, Exception? causa = null)
    : Exception(mensaje, causa);
```

Implementación funcional: `AceriaApp.Comunicacion.Contratos.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Contratos/AceriaApp.Comunicacion.Contratos.csproj`. Crea el archivo y sus carpetas si no existen.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
</ItemGroup>
</Project>
```

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add src\AceriaApp.Comunicacion.Contratos\AceriaApp.Comunicacion.Contratos.csproj
```

Implementación funcional: `MaterialDto.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Contratos/MaterialDto.cs`. Crea el archivo y sus carpetas si no existen.

```
namespace AceriaApp.Comunicacion.Contratos;
public sealed record MaterialDto(string Material, int Stock);
```

PASO 6.1.6 — Crear la clase `LlamadaSincrona`

LlamadaSincrona contiene información sobre el origen, el destino, el método y las políticas esperadas. Es una descripción de diseño: sus propiedades no ponen en marcha un timeout, un reintento ni un circuit breaker.

1. Crea `src/AceriaApp.Api/ComunicacionSincrona/LlamadaSincrona.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/LlamadaSincrona.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ComunicacionSincrona;

public class LlamadaSincrona
{
    public string ServicioOrigen { get; init; } = string.Empty;
    public string ServicioDestino { get; init; } = string.Empty;
    public string Metodo { get; init; } = string.Empty;
    public bool TieneTimeout { get; init; }
    public bool TieneReintentos { get; init; }
    public bool TieneCircuitBreaker { get; init; }
    public bool TieneClaveIdempotencia { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `LlamadaSincrona` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.ComunicacionSincrona` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.1.7 — Crear la clase `ValidadorComunicacionSincrona`

El validador comprueba si una descripción cumple las reglas declaradas para una llamada. Sus resultados no indican que Inventario esté disponible; esa garantía se verifica ejecutando peticiones reales entre los procesos.

1. Crea `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ComunicacionSincrona;

public class ValidadorComunicacionSincrona
{
    private static readonly string[] MetodosPermitidos = { "GET", "HEAD", "OPTIONS", "POST", "PUT", "PATCH",
"DELETE" };

    public ResultadoComunicacionSincrona Validar(IEnumerable<LlamadaSincrona> llamadas)
    {
        var errores = new List<string>();
        var lista = llamadas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay llamadas declaradas.");
        }

        foreach (var llamada in lista)
        {
            if (string.IsNullOrWhiteSpace(llamada.ServicioOrigen))
            {
                errores.Add("Una llamada no declara el servicio origen.");
            }

            if (string.IsNullOrWhiteSpace(llamada.ServicioDestino))
            {
                errores.Add($"La llamada desde '{llamada.ServicioOrigen}' no declara el servicio destino.");
            }

            if (llamada.ServicioOrigen == llamada.ServicioDestino)
            {
                errores.Add($"La llamada de '{llamada.ServicioOrigen}' apunta al mismo servicio.");
            }

            if (!MetodosPermitidos.Contains(llamada.Metodo))
            {
                errores.Add($"Método no permitido en '{llamada.ServicioOrigen}': '{llamada.Metodo}'.");
            }

            if (!llamada.TieneTimeout)
            {
                errores.Add($"La llamada desde '{llamada.ServicioOrigen}' no tiene timeout.");
            }

            if (llamada.TieneReintentos && (llamada.Metodo is "POST" or "PATCH")
                && !llamada.TieneClaveIdempotencia)
            {
                errores.Add($"La llamada desde '{llamada.ServicioOrigen}' declara reintentos no seguros sin
clave de idempotencia.");
            }
        }

        return new ResultadoComunicacionSincrona(errores.Count == 0, errores);
    }
}
```

```
public record ResultadoComunicacionSincrona(bool EsValida, IReadOnlyList<string> Errores);
```

Limita los valores de texto reconocidos por la política didáctica. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Completa la condición anterior; solo rechaza cuando también se cumple este término. Devuelve validez y diagnósticos; no ejecuta ninguna comunicación remota.

Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorComunicacionSincrona` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.ComunicacionSincrona` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.1.8 — Añadir los `using` necesarios en `ValidadorComunicacionSincrona`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs`.
2. Inserta el bloque de importaciones inmediatamente antes del namespace; comprueba que el ancla aparece una sola vez.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `namespace AceriaApp.Api.ComunicacionSincrona;`. Debe aparecer una sola vez.

```
using System.Collections.Generic;
using System.Linq;
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorComunicacionSincrona.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Implementación funcional: `AceriaApp.Comunicacion.Infraestructura.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/AceriaApp.Comunicacion.Infraestructura.csproj`. Crea el archivo y sus carpetas si no existen.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Comunicacion.Aplicacion/
AceriaApp.Comunicacion.Aplicacion.csproj" />
    <ProjectReference Include="../AceriaApp.Produccion.Infraestructura/
AceriaApp.Produccion.Infraestructura.csproj" />
    <PackageReference Include="Microsoft.Extensions.Http" Version="8.0.1" />
</ItemGroup>
</Project>
```

Fija la versión de la dependencia para reproducir la compilación y sus API.

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add
src\AceriaApp.Comunicacion.Infraestructura\AceriaApp.Comunicacion.Infraestructura.csproj
```

Implementación funcional: `ClienteInventarioHttp.cs`

Este adaptador convierte HTTP en el puerto de consulta. Un 404 significa ausencia; un 500, un timeout o JSON incorrecto son una indisponibilidad técnica. La consulta no reserva stock.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/ClienteInventarioHttp.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Contratos;
namespace AceriaApp.Comunicacion.Infraestructura;
public sealed class ClienteInventarioHttp(HttpClient http) : IClienteInventario
{
    public async Task<MaterialDto?> ObtenerAsync(string material, CancellationToken ct)
    {
        ct.ThrowIfCancellationRequested();
        if (string.IsNullOrWhiteSpace(material)) throw new ArgumentException("Material obligatorio.",
nameof(material));
        try
        {
            using var respuesta = await http.GetAsync("api/v1/materiales/" + Uri.EscapeDataString(material),
ct);
            if (respuesta.StatusCode == HttpStatusCode.NotFound) return null;
            respuesta.EnsureSuccessStatusCode();
            var dto = await respuesta.Content.ReadFromJsonAsync<MaterialDto>(cancellationToken: ct);
            if (dto is null || string.IsNullOrWhiteSpace(dto.Material) || dto.Stock < 0)
                throw new InventarioNoDisponibleException("Inventario devolvió un contrato inválido.");
            return dto;
        }
        catch (OperationCanceledException) when (ct.IsCancellationRequested) { throw; }
        catch (Exception ex) when (ex is HttpRequestException or JsonException or OperationCanceledException)
        { throw new InventarioNoDisponibleException("No se pudo consultar Inventario.", ex); }
    }
}
```

Detiene la operación si el llamador ya la canceló, antes de producir efectos. Codifica el material como un segmento de URL, evitando que sus caracteres cambien la consulta. Hace observable un estado HTTP de error; solo el 404 tratado antes representa ausencia.

Deserializa el contrato remoto y propaga el token de cancelación.

PASO 6.1.9 — Crear la clase `CatalogoLlamadas`

El catálogo ofrece distintos ejemplos para analizar riesgos de una llamada. Editar esos datos no cambia el HttpClient de la aplicación ni su dirección de destino.

1. Crea `src/AceriaApp.Api/ComunicacionSincrona/CatalogoLlamadas.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/CatalogoLlamadas.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ComunicacionSincrona;

public class CatalogoLlamadas
{
    public static List<LlamadaSincrona> LlamadasPorDefecto()
    {
        return new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "Ordenes.Api", ServicioDestino = "Inventario.Api", Metodo = "GET",
TieneTimeout = true, TieneReintentos = true, TieneCircuitBreaker = true },
            new() { ServicioOrigen = "Ordenes.Api", ServicioDestino = "Clientes.Api", Metodo = "GET",
TieneTimeout = true, TieneReintentos = true, TieneCircuitBreaker = true },
            new() { ServicioOrigen = "Facturacion.Api", ServicioDestino = "Ordenes.Api", Metodo = "GET",
TieneTimeout = true, TieneReintentos = true, TieneCircuitBreaker = true }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Construye una entrada del catálogo o del caso de prueba con valores explícitos. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoLlamadas` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.ComunicacionSincrona` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.1.10 — Añadir el `using` necesario en `CatalogoLlamadas`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/ComunicacionSincrona/CatalogoLlamadas.cs`.
2. Inserta el bloque de importaciones inmediatamente antes del namespace; comprueba que el ancla aparece una sola vez.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/CatalogoLlamadas.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `namespace AceriaApp.Api.ComunicacionSincrona;`. Debe aparecer una sola vez.

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoLlamadas.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Implementación funcional: `AceriaApp.Inventario.Api.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj`. Crea el archivo y sus carpetas si no existen.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Comunicacion.Contratos/AceriaApp.Comunicacion.Contratos.csproj" /
>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
</ItemGroup>
</Project>
```

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add src\AceriaApp.Inventario.Api\AceriaApp.Inventario.Api.csproj
```

Implementación funcional: `InventarioDbContext.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/InventarioDbContext.cs`. Crea el archivo y sus carpetas si no existen.

```
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class Existencia
{
    public string Material { get; set; } = "";
    public int Stock { get; set; }
}
public sealed class InventarioDbContext(DbContextOptions<InventarioDbContext> opciones) : DbContext(opciones)
{
    public DbSet<Existencia> Existencias => Set<Existencia>();
    protected override void OnModelCreating(ModelBuilder model)
    {
        model.Entity<Existencia>().HasKey(x => x.Material);
        model.Entity<Existencia>().Property(x => x.Material).HasMaxLength(100);
```

```
    }
}
```

Implementación funcional: `InventarioProgram.cs`

Este ejecutable es un segundo proceso ASP.NET Core con su propio puerto y base SQL. No lee directamente tablas de Producción. Inicializa existencias de laboratorio y expone consultas; cuando se habilita, aloja el consumidor de eventos.

Archivo completo: `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Contratos;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        var app = builder.Build();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapGet("/api/v1/materiales/{material}", async (string material, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        await app.RunAsync();
    }
}
```

Mantiene SQL Server real como proveedor; no se sustituye por almacenamiento en memoria. Crea un ámbito por iteración o mensaje; el DbContext no se comparte entre trabajos concurrentes. Inicializa una base didáctica nueva. No migra un esquema existente: usa los nombres separados de cada etapa.

Guarda las existencias iniciales del laboratorio en la base propia de Inventario. Publica una consulta HTTP con un contrato de respuesta verificable. Obtiene datos para lectura sin incorporarlos al seguimiento de cambios.

PASO 6.1.11 — Crear el proyecto de tests del punto 6.1

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.ComunicacionSincrona.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 14 y 15 incluyen las clases de prueba completas.

3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj` y `dotnet restore tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj`.
Operación: Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.
- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 6.1.12 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj` y `dotnet build tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

PASO 6.1.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj` y `dotnet list tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj package`.
4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

Implementación funcional: `AceriaApp.Api.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/AceriaApp.Api.csproj`. Sustituye el contenido de la copia actual.

```
<Project Sdk="Microsoft.NET.Sdk.Web"><PropertyGroup><TargetFramework>net8.0</
TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</ImplicitUsings></
PropertyGroup><ItemGroup><PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
```

```
Version="8.0.21" /><PackageReference Include="FluentValidation.DependencyInjectionExtensions"
Version="11.11.0" /><PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" /></ItemGroup>
<ItemGroup>
    <ProjectReference Include="../AceriaApp.Hexagonal.Aplicacion/AceriaApp.Hexagonal.Aplicacion.csproj" />
    <ProjectReference Include="../AceriaApp.Hexagonal.Infraestructura/
AceriaApp.Hexagonal.Infraestructura.csproj" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Produccion.Infraestructura/
AceriaApp.Produccion.Infraestructura.csproj" />
  </ItemGroup>
  <ItemGroup><ProjectReference Include="../AceriaApp.Comunicacion.Infraestructura/
AceriaApp.Comunicacion.Infraestructura.csproj" /></ItemGroup>
</Project>
```

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add src\AceriaApp.Api\AceriaApp.Api.csproj
```

Implementación funcional: `ComunicacionEndpoints.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/Comunicacion/ComunicacionEndpoints.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Aplicacion;
namespace AceriaApp.Api.Comunicacion;
public static class ComunicacionEndpoints
{
    public static void MapComunicacion(this WebApplication app)
    {
        app.MapGet("/api/v1/comunicacion/materiales/{material}", async (string material, IClienteInventario
inventario, CancellationToken ct) =>
        {
            try
            {
                var dto = await inventario.ObtenerAsync(material, ct);
                return dto is null ? Results.NotFound() : Results.Ok(dto);
            }
            catch (InventarioNoDisponibleException)
            { return Results.Problem(statusCode: 503, title: "Inventario no está disponible."); }
        }).WithTags("Comunicacion");
    }
}
```

Implementación funcional: `Program.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/Program.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Api.Comunicacion;
using AceriaApp.Api.Produccion;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.Api.Patrones;
using AceriaApp.Hexagonal.Infraestructura.Patrones;
using AceriaApp.Api.Datos;
using AceriaApp.Api.Modelo;
using Microsoft.EntityFrameworkCore;
using AceriaApp.Api.Contratos;
using AceriaApp.Api.Validacion;
using FluentValidation;
```

```
using AceriaApp.Api.Documentacion;

using AceriaApp.Api.Hexagonal;
using AceriaApp.Hexagonal.Infraestructura;
using AceriaApp.Hexagonal.Infraestructura.Datos;
var builder=WebApplication.CreateBuilder(args);
// INICIO PASO 2.1.12: SQL Server Express LocalDB, no SQLite.
builder.Services.AddDbContext<AceriaDbContext>(o=>
    o.UseSqlServer(builder.Configuration.GetConnectionString("AceriaApp")));
// PASO 2.3.14: registrar validadores en el contenedor.
builder.Services.AddValidatorsFromAssemblyContaining<CrearOrdenValidator>(); // PASO 2.3.14
// PASO 2.4.6: registrar generador antes de Build.
builder.Services.AddDocumentacion(); // PASO 2.4.6

builder.Services.AddServiciosHexagonales(builder.Configuration);
builder.Services.AddPatronesOrdenes();
builder.Services.AddProduccion(builder.Configuration);
builder.Services.AddComunicacion(builder.Configuration);
var app=builder.Build();
// EnsureCreated se utiliza SOLO como ayuda didáctica en Development.
// Para un sistema productivo se utilizarían migraciones controladas.
// PASO 2.1.13: inicializar LocalDB solo en Development.
if(app.Environment.IsDevelopment()&&!app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope=app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<AceriaDbContext>().Database.EnsureCreatedAsync();
}

// INICIO PASO 2.1.14: colección de recursos, consulta real con EF Core.
app.MapGet("/api/v1/ordenes",async (AceriaDbContext db,CancellationToken ct)=>
{
    // PASO 2.1.15: lectura sin tracking y proyeccion a DTO.
    var ordenes=await db.Ordenes.AsNoTracking().OrderBy(x=>x.FechaCreacion).ToListAsync(ct);
    return Results.Ok(ordenes.Select(OrdenDto.Desde));
})
.WithName("ListarOrdenes").WithTags("Ordenes")
.WithSummary("Lista las órdenes de producción").Produces<IEnumerable<OrdenDto>>(200)
;
// INICIO PASO 2.1.17: recurso individual.
app.MapGet("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(x=>x.Id==id);
    // PASO 2.1.18: 200 para orden presente; 404 si no existe.
    IResult response=orden is null ? Results.NotFound() : Results.Ok(OrdenDto.Desde(orden));
    return response;
})
.WithName("ObtenerOrdenPorId").WithTags("Ordenes")
.WithSummary("Obtiene una orden por ID").Produces<OrdenDto>(200).Produces(404)
;


// INICIO PASO 2.2.5: crear una orden, persistirla, devolver 201 + Location.
app.MapPost("/api/v1/ordenes",async (CrearOrdenRequest request,AceriaDbContext db)=>
{
    if(request.Cantidad>500)
        return Results.Problem(title:"El taller solo admite lotes de hasta 500.",
            statusCode:StatusCodes.Status422UnprocessableEntity);
    // PASO 2.2.6: crear entidad y persistirla.
    var orden=new Orden{Cliente=request.Cliente,Material=request.Material,Cantidad=request.Cantidad};
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    // PASO 2.2.7: devolver 201 con Location.
    return Results.CreatedAtRoute("ObtenerOrdenPorId",new{id=orden.Id},OrdenDto.Desde(orden));
})
.WithName("CrearOrden").WithTags("Ordenes")
// PASO 2.3.15: filtro de POST.
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Crea una orden").Produces<OrdenDto>(201).ProducesValidationProblem(400).ProducesProblem(422)
;


// INICIO PASO 2.2.11: sustitución completa de los campos editables.
app.MapPut("/api/v1/ordenes/{id:guid}",async (Guid id,CrearOrdenRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
```

```
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cliente=request.Cliente;orden.Material=request.Material;
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("ReemplazarOrden").WithTags("Ordenes")
.AddEndpointFilter<ValidationFilter<CrearOrdenRequest>>()
.WithSummary("Sustituye campos
editables").Produces(204).Produces(404).ProducesProblem(409).ProducesValidationProblem(400).ProducesProblem(4
22)
;


// INICIO PASO 2.2.14: cambiar SOLO la cantidad.
app.MapPatch("/api/v1/ordenes/{id:guid}/cantidad",
    async (Guid id,ActualizarCantidadRequest request,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede modificar una orden finalizada.",statusCode:409);
    if(request.Cantidad>500) return Results.Problem(
        title:"El taller solo admite lotes de hasta 500.",statusCode:422);
    orden.Cantidad=request.Cantidad;
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("CambiarCantidad").WithTags("Ordenes")
// PASO 2.3.17: filtro de PATCH.
.AddEndpointFilter<ValidationFilter<ActualizarCantidadRequest>>()
.WithSummary("Modifica
cantidad").Produces<OrdenDto>(200).Produces(404).ProducesValidationProblem(400).ProducesProblem(409).Produces
Problem(422)
;


// INICIO PASO 2.2.16: eliminar recurso persistido.
app.MapDelete("/api/v1/ordenes/{id:guid}",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"No se puede eliminar una orden finalizada.",statusCode:409);
    db.Ordenes.Remove(orden);
    await db.SaveChangesAsync();
    return Results.NoContent();
})
.WithName("EliminarOrden").WithTags("Ordenes")
.WithSummary("Elimina una orden pendiente").Produces(204).Produces(404).ProducesProblem(409)
;
// INICIO PASO 2.2.18: transición explícita del dominio.
app.MapPost("/api/v1/ordenes/{id:guid}/finalizar",async (Guid id,AceriaDbContext db)=>
{
    var orden=await db.Ordenes.FindAsync(id);
    if(orden is null) return Results.NotFound();
    if(orden.Estado=="Finalizada") return Results.Problem(
        title:"La orden ya estaba finalizada.",statusCode:409);
    orden.Estado="Finalizada";
    await db.SaveChangesAsync();
    return Results.Ok(OrdenDto.Desde(orden));
})
.WithName("FinalizarOrden").WithTags("Ordenes")
.WithSummary("Finaliza una orden").Produces<OrdenDto>(200).Produces(404).ProducesProblem(409)
;

// PASO 2.4.8: habilitar Swagger solo en Development.
app.UseDocumentacion(); // PASO 2.4.8
// El contexto hexagonal usa su propia base para que EnsureCreated no dependa de
// las tablas heredadas del curso M02.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
```

```
    await scope.ServiceProvider.GetRequiredService<HexagonalDbContext>()
        .Database.EnsureCreatedAsync();
}
app.MapHexagonalOrdenes();
app.MapPatronesOrdenes();
// Base didáctica propia del contexto Producción, independiente de los modelos anteriores.
if (app.Environment.IsDevelopment() && !app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
{
    using var scope = app.Services.CreateScope();
    await scope.ServiceProvider.GetRequiredService<ProduccionDbContext>().Database.EnsureCreatedAsync();
}
app.MapProduccion();
app.MapComunicacion();
app.Run();
public partial class Program {} // permite WebApplicationFactory en tests
```

Implementación funcional: `appsettings.json`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/appsettings.json`. Sustituye el contenido de la copia actual.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM061;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM061;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM061;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m061",
    "WorkersActivos": false
  }
}
```

Separa los almacenes de esta etapa y conserva los de los módulos anteriores.

Implementación funcional: `RegistroComunicacion.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/RegistroComunicacion.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Aplicacion;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
namespace AceriaApp.Comunicacion.Infraestructura;
public static class RegistroComunicacion
{
    public static IServiceCollection AddComunicacion(this IServiceCollection services, IConfiguration config)
    {
        services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>(http =>
        {
            http.BaseAddress = new Uri(config["Comunicacion:InventarioUrl"] ?? "http://localhost:5106/");
            http.Timeout = TimeSpan.FromSeconds(3);
        });
        return services;
    }
}
```

Registra el cliente tipado y centraliza dirección y timeout; los handlers se administran mediante la factoría.

Implementación funcional: `appsettings.json`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/appsettings.json`. Crea el archivo y sus carpetas si no existen.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM061;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m061",
    "WorkersActivos": false
  }
}
```

PASO 6.1.14 — Crear el archivo `ValidadorComunicacionSincronaTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.ComunicacionSincrona.Tests/ValidadorComunicacionSincronaTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.ComunicacionSincrona.Tests/ValidadorComunicacionSincronaTests.cs`.
Operación: Crear.

```
using AceriaApp.Api.ComunicacionSincrona;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ComunicacionSincrona.Tests;

public class ValidadorComunicacionSincronaTests
{
    private readonly ValidadorComunicacionSincrona _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoLlamadas.LlamadasPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinLlamadas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<LlamadaSincrona>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay llamadas"));
```

```
    }

    [Fact]
    public void Validar_MismoOrigenYDestino_DevuelveError()
    {
        var llamadas = new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "Ordenes.Api", ServicioDestino = "Ordenes.Api", Metodo = "GET",
TieneTimeout = true, TieneReintentos = true }
        };

        var resultado = _validador.Validar(llamadas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("apunta al mismo servicio"));
    }

    [Fact]
    public void Validar_MetodoNoPermitido_DevuelveError()
    {
        var llamadas = new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "A", ServicioDestino = "B", Metodo = "CONNECT", TieneTimeout = true,
TieneReintentos = true }
        };

        var resultado = _validador.Validar(llamadas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Método no permitido"));
    }

    [Fact]
    public void Validar_SinTimeout_DevuelveError()
    {
        var llamadas = new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "A", ServicioDestino = "B", Metodo = "GET", TieneTimeout = false,
TieneReintentos = true }
        };

        var resultado = _validador.Validar(llamadas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene timeout"));
    }

    [Fact]
    public void Validar_PostSinReintentos_DevuelveValido()
    {
        var llamadas = new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "A", ServicioDestino = "B", Metodo = "POST", TieneTimeout = true,
TieneReintentos = false }
        };

        var resultado = _validador.Validar(llamadas);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
    [Fact]
    public void Validar_PostConReintentosSinIdempotencia_DevuelveError()
    {
        var llamadas = new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "A", ServicioDestino = "B", Metodo = "POST", TieneTimeout = true,
TieneReintentos = true }
        };

        var resultado = _validador.Validar(llamadas);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("reintentos no seguros"));
    }
```

```
    [Fact]
    public void Validar_PostConReintentosYClaveIdempotente_DevuelveValido()
    {
        var llamadas = new List<LlamadaSincrona>
        {
            new() { ServicioOrigen = "A", ServicioDestino = "B", Metodo = "POST", TieneTimeout = true,
TieneReintentos = true, TieneClaveIdempotencia = true }
        };

        var resultado = _validador.Validar(llamadas);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Verificación concreta. `ValidadorComunicacionSincronaTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 6.1.15 — Crear el archivo `CatalogoLlamadasTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.ComunicacionSincrona.Tests/CatalogoLlamadasTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.ComunicacionSincrona.Tests/CatalogoLlamadasTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.ComunicacionSincrona;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ComunicacionSincrona.Tests;

public class CatalogoLlamadasTests
{
    [Fact]
    public void LlamadasPorDefecto_ContieneTresLlamadas()
    {
        var llamadas = CatalogoLlamadas.LlamadasPorDefecto();

        llamadas.Should().HaveCount(3);
    }

    [Fact]
    public void LlamadasPorDefecto_TodasUsanGet()
    {
```

```
        var llamadas = CatalogoLlamadas.LlamadasPorDefecto();

        llamadas.Should().OnlyContain(l => l.Metodo == "GET");
    }

    [Fact]
    public void LlamadasPorDefecto_TodasTienenTimeout()
    {
        var llamadas = CatalogoLlamadas.LlamadasPorDefecto();

        llamadas.Should().OnlyContain(l => l.TieneTimeout);
    }

    [Fact]
    public void LlamadasPorDefecto_TodasTienenReintentos()
    {
        var llamadas = CatalogoLlamadas.LlamadasPorDefecto();

        llamadas.Should().OnlyContain(l => l.TieneReintentos);
    }

    [Fact]
    public void LlamadasPorDefecto_NingunaApuntaASiMisma()
    {
        var llamadas = CatalogoLlamadas.LlamadasPorDefecto();

        llamadas.Should().OnlyContain(l => l.ServicioOrigen != l.ServicioDestino);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Comprueba la condición en cada entrada declarada del catálogo.

Verificación concreta. `CatalogoLlamadasTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Implementación funcional: `AceriaApp.ComunicacionSincrona.Tests.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj`. Sustituye el contenido de la copia actual.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
</ItemGroup>
<ItemGroup><Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" /></ItemGroup>
</Project>
```

Implementación funcional: `ClienteInventarioTests.cs`

Un HttpMessageHandler controlado permite provocar respuestas y cancelaciones concretas en el cliente real. Estas pruebas no acreditan conectividad de red; el recorrido de dos procesos verifica esa parte.

Archivo completo: `tests/AceriaApp.ComunicacionSincrona.Tests/ClienteInventarioTests.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Net;
using System.Text;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Infraestructura;
using Xunit;
namespace AceriaApp.ComunicacionSincrona.Tests;
public class ClienteInventarioTests
{
    private sealed class Servidor(Func<HttpRequestMessage,CancellationToken,Task<HttpResponseMessage>>
respuesta) : HttpMessageHandler
    {
        protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage r,CancellationToken ct) =>
respuesta(r,ct);
    }
    private static ClienteInventarioHttp Cliente(HttpStatusCode estado,string json) =>
        new(new HttpClient(new Servidor((_,_)=>Task.FromResult(new HttpResponseMessage(estado)
        {Content=new StringContent(json,Encoding.UTF8,"application/json")}))) {BaseAddress=new Uri("http://
inventario/")});
    [Fact, Trait("Tipo","Unidad")]
    public async Task Json_valido_se_traduce_al_contrato()
    { var r=await Cliente(HttpStatusCode.OK,"{\"material\":\"Acero\",
\"stock\":80}").ObtenerAsync("Acero",default); Assert.Equal(80,r!.Stock); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Solo_404_se_traduce_a_ausencia()
    { Assert.Null(await Cliente(HttpStatusCode.NotFound,"{}").ObtenerAsync("Acero",default)); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Error_500_no_se_disfraza_de_material_ausente()
    { await
Assert.ThrowsAsync<InventarioNoDisponibleException>(()=>Cliente(HttpStatusCode.InternalServerError,"{}").Obte
nerAsync("Acero",default)); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Json_invalido_es_fallo_del_servicio()
    { await
Assert.ThrowsAsync<InventarioNoDisponibleException>(()=>Cliente(HttpStatusCode.OK,"{").ObtenerAsync("Acero",d
efault)); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Json_nulo_no_es_404()
    { await
Assert.ThrowsAsync<InventarioNoDisponibleException>(()=>Cliente(HttpStatusCode.OK,"null").ObtenerAsync("Acero
",default)); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Stock_negativo_es_contrato_invalido()
    { await Assert.ThrowsAsync<InventarioNoDisponibleException>(()=>Cliente(HttpStatusCode.OK,"{\"material\":
\"Acero\",\"stock\":-1}").ObtenerAsync("Acero",default)); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Cancelacion_del_llamador_se_propaga()
    { using var ct=new CancellationTokenSource();ct.Cancel();await
Assert.ThrowsAnyAsync<OperationCanceledException>(()=>Cliente(HttpStatusCode.OK,"{}").ObtenerAsync("Acero",ct
.Token)); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Ruta_escapa_espacios_y_no_inyecta_query()
    {
        string? ruta=null;
        using var http=new HttpClient(new Servidor((r,_)=>{ruta=r.RequestUri!.AbsoluteUri;return
Task.FromResult(new HttpResponseMessage(HttpStatusCode.NotFound));})){BaseAddress=new Uri("http://
inventario/")};
        await new ClienteInventarioHttp(http).ObtenerAsync("Acero ?x=1",default);
        Assert.Equal("http://inventario/api/v1/materiales/Acero%20%3Fx%3D1",ruta);
    }
}
```

Comprueba una consecuencia observable. Si cambia el comportamiento descrito por el nombre del test, esta aserción debe fallar.

Implementación funcional: `InventarioSqlTests.cs`

La factoría aloja la API de Inventario para comprobar contratos HTTP usando SQL Server real. Este alojamiento de prueba se complementa con el recorrido entre dos procesos Kestrel descrito al final del punto.

Archivo completo: `tests/AceriaApp.ComunicacionSincrona.Tests/InventarioSqlTests.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Net;
using System.Net.Http.Json;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Inventario.Api;
using AceriaApp.PruebasSqlServer;
using Microsoft.AspNetCore.Hosting;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.AspNetCore.TestHost;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.DependencyInjection.Extensions;
using Xunit;
namespace AceriaApp.ComunicacionSincrona.Tests;
public class InventarioSqlTests
{
    private sealed class Factory : WebApplicationFactory<InventarioProgram>
    {
        private readonly BaseSqlServer bd=BaseSqlServer.Crear();
        public Factory()
        {
            using var db=Nuevo();db.Database.EnsureCreated();
            db.Existencias.Add(new Existencia{Material="Acero",Stock=80});db.SaveChanges();
        }
        public InventarioDbContext Nuevo()=>new(new
DbContextOptionsBuilder<InventarioDbContext>().UseSqlServer(bd.ConnectionString).Options);
        protected override void ConfigureWebHost(IWebHostBuilder builder)
        {
            builder.UseEnvironment("Development");builder.UseSetting("SkipDatabaseInitialization","true");

builder.ConfigureTestServices(s=>{s.RemoveAll<DbContextOptions<InventarioDbContext>>();s.AddDbContext<Inventa
rioDbContext>(o=>o.UseSqlServer(bd.ConnectionString));});
        }
        protected override void Dispose(bool disposing){base.Dispose(disposing);if(disposing)bd.Dispose();}
    }
    [Fact]
    public async Task API_devuelve_datos_persistidos_con_SQLServer()
    {using var f=new Factory();using var http=f.CreateClient();var r=await
http.GetFromJsonAsync<MaterialDto>("/api/v1/materiales/Acero");Assert.Equal(80,r!.Stock);using var
db=f.Nuevo();Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer",db.Database.ProviderName);}
    [Fact]
    public async Task API_devuelve_404_para_material_ausente()
    {using var f=new Factory();using var http=f.CreateClient();Assert.Equal(HttpStatusCode.NotFound,(await
http.GetAsync("/api/v1/materiales/NoExiste")).StatusCode);}
    [Fact]
    public async Task Stock_se_recupera_desde_otro_contexto_SQL()
    {using var f=new Factory();using(var db=f.Nuevo()){(await db.Existencias.SingleAsync()).Stock=70;await
db.SaveChangesAsync();}using var otro=f.Nuevo();Assert.Equal(70,(await
otro.Existencias.SingleAsync()).Stock);}
}
```

Libera recursos del adaptador o de la prueba al terminar su ámbito. Guarda los datos de preparación o la modificación que esta prueba comprobará desde SQL Server real.

PASO 6.1.16 — Compilar la solución completa

La compilación acredita que se resuelven los nuevos proyectos y contratos, pero no demuestra que los dos procesos puedan comunicarse. El ensayo funcional debe iniciar Producción e Inventario y comprobar sus respuestas.

1. Aplica los bloques de integración funcional de este paso en el orden mostrado.

2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 27 proyectos y termina sin errores.

Los archivos funcionales de este punto se han incorporado en los pasos 5, 8, 9, 10, 13 y 15. Antes de compilar, agrega todos los proyectos nuevos a la solución con los comandos indicados y guarda los archivos completos.

Verificación concreta. La solución de la etapa compila con 27 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

PASO 6.1.17 — Ejecutar todos los tests del punto 6.1

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release --no-build --no-restore`.
2. Comprueba que se ejecutan los 13 tests de catálogo y las pruebas funcionales añadidas al proyecto hasta el paso 15.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 13 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 6.1.18 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.
3. Suma los resultados de los archivos TRX y verifica 232 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04 y M05: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 232 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

Comprobación con procesos reales

Utiliza las dos terminales y las direcciones del apartado «Arranque de los dos procesos». Comprueba que Inventario escucha en 5106 y Producción en 5105. Los tests de cliente con un handler controlado y los de WebApplicationFactory no reemplazan este recorrido por sockets.

PASO 6.1.19 — Actualizar el README del punto 6.1

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `6.1-comunicacion-sincrona-http`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `5.5-dominio-aceria` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 6.1 — Comunicación síncrona mediante HTTP

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base
anterior.

## Estado del proyecto

- Solución `AceriaApp.sln`: 27 proyectos y 232 pruebas acumuladas.
- 13 pruebas nuevas de análisis en `tests/AceriaApp.ComunicacionSincrona.Tests`.
- Tipos `LlamadaSincrona`, `ValidadorComunicacionSincrona` y `CatalogoLlamadas` en `src/AceriaApp.Api/
ComunicacionSincrona`.
- Los catálogos no demuestran tráfico HTTP ni entrega de mensajes.

## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M06 para iniciar los adaptadores y observar cada recorrido de comunicación.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/comunicacion-sincrona-http.md` y el ADR-024. Los bloques
funcionales de los pasos 5 a 15 completan la integración del punto. Se conservan los documentos y pruebas
heredados.

## Documentación

- [Tema del punto](docs/comunicacion-sincrona-http.md).
- [ADR-024](docs/ADR-024-comunicacion-sincrona-http.md).
- Los 28 documentos anteriores, incluidos los ADR-010 a ADR-023, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 27 proyectos, 232 pruebas y 13 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 6.1.20 — Crear el ADR-024 del punto 6.1

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-024-comunicacion-sincrona-http.md` dentro de la copia de `6.1-comunicacion-sincrona-http`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-024-comunicacion-sincrona-http.md` existe y que los 28 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-024-comunicacion-sincrona-http.md`. **Operación:** Crear.

```
# ADR-024: Comunicación síncrona mediante HTTP

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita razonar explícitamente sobre sus interacciones. La solución conserva los modelos y pruebas heredados; el catálogo nuevo es un ejercicio de análisis separado de los adaptadores de comunicación ejecutables.

## Decisión

Registrar las llamadas, validar destinos, métodos y timeout, y revisar la seguridad de los reintentos. No exigir reintentos para todas las operaciones: POST y PATCH necesitan un contrato de idempotencia antes de repetirse. El cliente HTTP y las pruebas de tráfico verifican por separado el comportamiento real.

## Alternativas consideradas

- Elegir un solo estilo para todo el sistema: descartado porque las interacciones tienen necesidades diferentes.
- Omitir los criterios de recuperación y consistencia: descartado porque deja ambiguos los fallos parciales.
- Considerar suficiente la validación del catálogo: descartado porque no comprueba la comunicación ejecutable.

## Consecuencias

- Positivas: decisiones explícitas, casos de rechazo reproducibles y trazabilidad por interacción.
- Negativas: mantener contratos, políticas y pruebas de comportamiento añade trabajo.
- Mitigación: distinguir pruebas de análisis y pruebas funcionales, mantener límites claros y revisar las decisiones con evidencia operativa.
```

Verificación concreta. `docs/ADR-024-comunicacion-sincrona-http.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-024-comunicacion-sincrona-http.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 6.1.21 — Verificar que no se ha modificado ningún archivo del punto 5.5

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M05_DDD_CQRS/PROYECTO/5.5-dominio-aceria` y actual `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.

3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 28 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 28 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 6.1.22 — Verificar la estructura del punto 6.1

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, sin proyectos ocultos fuera de la etapa.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/ComunicacionSincrona` y las dos clases de tests en `tests/AceriaApp.ComunicacionSincrona.Tests`.
3. Comprueba los 30 documentos de `docs`, incluida la ficha del punto y el ADR-024.
4. Ejecuta `dotnet sln AceriaApp.sln list` y contrasta sus 27 proyectos con los `.csproj` presentes.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 27 proyectos.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 6.1.23 — Verificar el `.gitignore` del punto 6.1

Las exclusiones se aplican desde la raíz a las etapas; no es necesario inventar un `.gitignore` propio dentro de cada copia.

1. Abre el `.gitignore` de la raíz del repositorio del curso.
2. Comprueba las reglas `**/bin/`, `**/obj/` y `**/TestResults/` que ya excluyen los artefactos de todas las etapas.
3. Ejecuta `git status --short` y revisa que no aparezcan binarios, cachés ni resultados temporales.
4. Conserva las exclusiones existentes; si aparecen archivos locales de Visual Studio, excluye `.vs/`, `*.user` y `*.suo` mediante `.git/info/exclude`, sin ignorar código ni configuración entregable.

Verificación concreta. Los archivos compilados y resultados de tests están excluidos; los archivos del punto siguen visibles para Git.

Diagnóstico si falla.

- Ignorar todo `tests/` elimina la evidencia reproducible: excluye solo sus artefactos generados.
- Esperar un `.gitignore` dentro de cada etapa puede llevar a crear duplicados innecesarios: revisa primero el de la raíz.

PASO 6.1.24 — Hacer el commit del punto 6.1 y verificar el estado final

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 6.1: comunicación síncrona mediante http`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 6.1 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorComunicacionSincrona` y una prueba adicional. Trabaja en una copia optativa de `6.1-comunicacion-sincrona-http` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 6.1.1 — Definir el contrato del resumen

Abre `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs` e inserta el contrato después del `record` de resultado, en el mismo namespace.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs`. **Operación:** Insertar después del ancla indicada.

Ancla exacta: `public record ResultadoComunicacionSincrona(bool EsValida, IReadOnlyList<string> Errores);`. Debe aparecer una sola vez.

```
public record ResumenLlamadas(int Total, int ConTimeout, int ConReintentos, int ConCircuitBreaker);
```

Reto 6.1.3 — Calcular los recuentos

Inserta el método dentro de `ValidadorComunicacionSincrona`, antes del campo `private static readonly string[]`; conserva las cuatro posiciones del constructor del resumen.

Archivo: `src/AceriaApp.Api/ComunicacionSincrona/ValidadorComunicacionSincrona.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `private static readonly string[]`. Debe aparecer una sola vez.

```
    public static ResumenLlamadas Resumen(IEnumerable<LlamadaSincrona> llamadas)
    {
        var lista = llamadas.ToList();
        var conTimeout = lista.Count(l => l.TieneTimeout);
        var conReintentos = lista.Count(l => l.TieneReintentos);
        var conCircuitBreaker = lista.Count(l => l.TieneCircuitBreaker);
        return new ResumenLlamadas(lista.Count, conTimeout, conReintentos, conCircuitBreaker);
    }
```

Cuenta las entradas que cumplen el criterio indicado. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 6.1.6 — Verificar el catálogo por defecto

Inserta la prueba dentro de `ValidadorComunicacionSincronaTests`, antes del campo `_validador`. La prueba comprueba total 3, con timeout 3, con reintentos 3 y con circuit breaker 3.

Archivo: `tests/AceriaApp.ComunicacionSincrona.Tests/ValidadorComunicacionSincronaTests.cs`.
Operación: Insertar antes del ancla indicada.

Ancla exacta: `private readonly ValidadorComunicacionSincrona _validador = new();`. Debe aparecer una sola vez.

```
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorComunicacionSincrona.Resumen(CatalogoLlamadas.LlamadasPorDefecto());

        resumen.Total.Should().Be(3);
        resumen.ConTimeout.Should().Be(3);
        resumen.ConReintentos.Should().Be(3);
        resumen.ConCircuitBreaker.Should().Be(3);
    }
```

Calcula el resumen sobre el catálogo por defecto. Compara el recuento obtenido con el valor esperado del catálogo.

Comprobación del reto. Compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.ComunicacionSincrona.Tests/AceriaApp.ComunicacionSincrona.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 13 a 14 tests; la solución debe ejecutar 232 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que la etapa obligatoria conserva sus hashes.

Diagnóstico si falla. Si el método queda fuera de la clase, mueve solo el fragmento al ancla señalada. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 13 pruebas de catálogo, los adaptadores funcionales y 30 documentos Markdown. La solución incluye 27 proyectos y 232 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.

Punto 6.2 — Comunicación asíncrona basada en eventos

Comunicar hechos mediante eventos y comprobar su recepción

Cuando un servicio anuncia un hecho de negocio, el destinatario no siempre tiene que estar disponible en ese mismo instante. RabbitMQ permite desacoplar temporalmente la publicación y el consumo, pero obliga a tratar explícitamente fallos, reentregas y la posibilidad de efectos duplicados. En esta etapa distinguiremos guardar la operación, publicar el evento y confirmar que el consumidor aplicó su cambio.

El proyecto conserva el recorrido HTTP anterior y añade publicación, consumidores, outbox e inbox. RabbitMQ se ejecutará en Docker Desktop, mientras las API siguen funcionando como procesos Windows. Las pruebas deben observar la cola y el estado persistido: crear un objeto llamado Evento o registrar un catálogo de mensajes no demuestra que el intercambio haya tenido lugar.

Carpeta de trabajo: `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar por separado el análisis y la comunicación ejecutable. La etapa termina con 264 pruebas en 28 proyectos.

Ejercicio guiado. Incorpora `Evento`, `ValidadorComunicacionAsincrona` y `CatalogoEventos`, dos documentos nuevos y 13 pruebas de catálogo. Los bloques de integración de los pasos 5 a 15 implementan el comportamiento remoto.

PASO 6.2.1 — Crear la carpeta del punto 6.2 a partir del punto 6.1

La segunda etapa conserva las llamadas HTTP que ya funcionan y añade una vía de eventos. Esta copia evita modificar la referencia 6.1 y permite atribuir cualquier fallo nuevo a la incorporación de la mensajería.

1. Localiza la etapa completa `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos` y copia en ella el contenido de `6.1-comunicacion-sincrona-http`; en Windows usa `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `6.2-comunicacion-asincrona-eventos`.
4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `6.1-comunicacion-sincrona-http` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `6.1-comunicacion-sincrona-http`: verifica la ruta antes de editar.

PASO 6.2.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `6.2-comunicacion-asincrona-eventos` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 27 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 27 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 6.2.3 — Verificar que la carpeta `docs` contiene los documentos del punto 6.1

El punto conserva 30 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 30 archivos Markdown.
2. Localiza los ADR-010 a ADR-024 y las fichas temáticas heredadas.
3. Compara nombres y contenido con `docs` de `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http`.
4. Comprueba que aún no existen `docs/comunicacion-asincrona-eventos.md` ni `docs/ADR-025-comunicacion-asincrona-eventos.md`.

Verificación concreta. Hay 30 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 6.2.4 — Crear el documento de comunicación asíncrona en `docs`

La ficha separa el análisis de los contratos y la evidencia ejecutable.

1. Crea `docs/comunicacion-asincrona-eventos.md` dentro de la copia de `6.2-comunicacion-asincrona-eventos`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/comunicacion-asincrona-eventos.md` existe y que los 30 documentos anteriores permanecen intactos.

Archivo: `docs/comunicacion-asincrona-eventos.md`. **Operación:** Crear.

```
# Comunicación asíncrona basada en eventos — Punto 6.2

## Definición

El emisor publica un hecho y no espera al procesamiento de cada consumidor para terminar su operación. La aceptación por el broker y la finalización del trabajo del consumidor son momentos diferentes.

## Evento

Describe un hecho pasado y contiene identificador estable, versión, instante y datos suficientes para interpretarlo. Un evento de dominio pertenece al modelo local; un evento de integración es un contrato público entre fronteras. No se publican directamente entidades de dominio.

## Publicadores, consumidores y broker

El publicador conoce el contrato y el broker; no necesita enumerar todos los consumidores. RabbitMQ enruta mensajes según exchanges, colas y bindings. Que un catálogo declare consumidores no crea ninguna cola ni suscripción.

## Desacoplamiento temporal

Una cola puede retener mensajes durante una caída del consumidor, según su durabilidad y política de expiración. El broker sigue siendo una dependencia y debe estar disponible cuando el productor intenta publicar o habrá que diferir el envío.

## Consistencia eventual

Los estados de los servicios pueden diferir mientras se procesan mensajes. Define qué operación está pendiente, cómo se consulta su avance y cómo se resuelven fallos permanentes. Eventual no significa un plazo garantizado.
```

```
## Entrega e idempotencia

La entrega al menos una vez puede producir duplicados. El consumidor identifica el mensaje y registra de
forma durable su efecto antes de confirmar la entrega. Un ACK prematuro puede perder trabajo. Una
confirmación del publicador indica aceptación del broker, no éxito de todos los consumidores.

## Reintentos y mensajes fallidos

Acota los reintentos y su demora; separa fallos transitorios de contratos inválidos. Usa una cola de fallidos
y un procedimiento de inspección o reproceso. La política del catálogo exige que los eventos de integración
del ejercicio declaren estas decisiones; no pretende imponerlas a todos los eventos de dominio.

## Transacción y publicación

Guardar datos y publicar son dos operaciones. El patrón outbox registra el cambio y la intención de
publicación en una transacción local. Un despachador vuelve a publicar pendientes; el consumidor debe seguir
tolerando duplicados.

## Aplicación al curso

`Evento`, `CatalogoEventos` y `ValidadorComunicacionAsincrona` validan cuatro contratos de integración y seis
suscripciones declaradas. No representan seis servicios distintos ni demuestran conexión con RabbitMQ. Los
eventos locales de dominio pueden existir sin suscriptores remotos y sin políticas de entrega de un broker.

## Correspondencia ejecutable

La implementación funcional de mensajería debe comprobar publicación, consumo, persistencia del efecto e
idempotencia. Los tests del catálogo solo demuestran coherencia de sus entradas; no sustituyen esa
comprobación.
```

Verificación concreta. `docs/comunicacion-asincrona-eventos.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/comunicacion-asincrona-eventos.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 6.2.5 — Crear la carpeta `ComunicacionAsincrona` en el proyecto API

ComunicacionAsincrona aloja las descripciones de eventos y sus reglas. La publicación y el consumo se ejecutan en los componentes que se conectan realmente con RabbitMQ, no en el catálogo.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `ComunicacionAsincrona` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/ComunicacionAsincrona` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `6.1-comunicacion-sincrona-http`.

Verificación concreta. Existe `src/AceriaApp.Api/ComunicacionAsincrona` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

Implementación funcional: `PuertosPlanificacion.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Aplicacion/PuertosPlanificacion.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Contratos;
namespace AceriaApp.Comunicacion.Aplicacion;
public enum EstadoPlanificacion { Aceptada, NoExiste, Conflicto, SinDisponibilidad }
public interface IPlanificadorIntegrado
{
    Task<IReadOnlyList<LineaPlanificadaV1>?> ObtenerLineasAsync(Guid id, CancellationToken ct);
    Task<EstadoPlanificacion> PlanificarAsync(Guid id, CancellationToken ct);
}
public interface ITransporteEventos
{
    Task PublicarAsync(string contenido, CancellationToken ct);
}
```

Define la forma pública o la configuración reproducible que usarán los archivos del mismo punto.

Implementación funcional: `OrdenPlanificadaV1.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Contratos/OrdenPlanificadaV1.cs`. Crea el archivo y sus carpetas si no existen.

```
namespace AceriaApp.Comunicacion.Contratos;
public sealed record LineaPlanificadaV1(string Material, int Cantidad);
public sealed record OrdenPlanificadaV1(Guid EventoId, Guid OrdenId, DateTimeOffset OcurridoEnUtc,
    IReadOnlyList<LineaPlanificadaV1> Lineas, int Version = 1)
{
    public void Validar()
    {
        if (Version != 1 || EventoId == Guid.Empty || OrdenId == Guid.Empty || OcurridoEnUtc == default ||
            Lineas is null || Lineas.Count == 0 || Lineas.Count > 500 ||
            Lineas.Any(x => x is null || string.IsNullOrWhiteSpace(x.Material) || x.Material.Length > 100 ||
x.Cantidad < 1 || x.Cantidad > 500) ||
            Lineas.Sum(x => (long)x.Cantidad) > 500)
            throw new InvalidDataException("Evento de planificación incompatible o inválido.");
    }
}
```

Utiliza el contrato público versionado, sin serializar la entidad de dominio completa. Señala un contrato incompatible o un conflicto de identidad; el consumidor lo envía a fallidos.

PASO 6.2.6 — Crear la clase `Evento`

Evento describe un hecho con información sobre productor y consumidor. Crear una instancia no equivale a publicarla; habrá que comprobar después que el broker recibe el mensaje.

1. Crea `src/AceriaApp.Api/ComunicacionAsincrona/Evento.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/Evento.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ComunicacionAsincrona;

public class Evento
{
    public string Nombre { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;
    public string Publicador { get; init; } = string.Empty;
```

```
    public int NumeroDeConsumidores { get; init; }
    public bool TieneIdempotencia { get; init; }
    public bool TieneReintentos { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `Evento` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.ComunicacionAsincrona` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.2.7 — Crear la clase `ValidadorComunicacionAsincrona`

La validez de un evento declarado no garantiza que se entregue ni que se procese una sola vez. El flujo real debe ejercitar el broker, el consumidor y la persistencia outbox/inbox.

1. Crea `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ComunicacionAsincrona;

public class ValidadorComunicacionAsincrona
{
    private static readonly string[] TiposPermitidos = { "Dominio", "Integracion" };

    public ResultadoComunicacionAsincrona Validar(IEnumerable<Evento> eventos)
    {
        var errores = new List<string>();
        var lista = eventos.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay eventos declarados.");
        }

        foreach (var evento in lista)
        {
            if (string.IsNullOrWhiteSpace(evento.Nombre))
            {
                errores.Add("Un evento no declara el nombre.");
            }

            if (!TiposPermitidos.Contains(evento.Tipo))
            {
                errores.Add($"Tipo no permitido en '{evento.Nombre}': '{evento.Tipo}'.");
            }

            if (string.IsNullOrWhiteSpace(evento.Publicador))
            {
                errores.Add($"El evento '{evento.Nombre}' no declara el publicador.");
            }

            if (evento.NumeroDeConsumidores < 0 || (evento.Tipo == "Integracion" &&
```

```
evento.NumeroDeConsumidores == 0))
            {
                errores.Add($"El evento '{evento.Nombre}' no tiene consumidores válidos para la política de
este catálogo.");
            }

            if (evento.Tipo == "Integracion" && !evento.TieneIdempotencia)
            {
                errores.Add($"El evento '{evento.Nombre}' no declara idempotencia.");
            }

            if (evento.Tipo == "Integracion" && !evento.TieneReintentos)
            {
                errores.Add($"El evento '{evento.Nombre}' no declara reintentos.");
            }
        }

        return new ResultadoComunicacionAsincrona(errores.Count == 0, errores);
    }
}

public record ResultadoComunicacionAsincrona(bool EsValida, IReadOnlyList<string> Errores);
```

Limita los valores de texto reconocidos por la política didáctica. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Devuelve validez y diagnósticos; no ejecuta ninguna comunicación remota. Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorComunicacionAsincrona` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.ComunicacionAsincrona` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.2.8 — Añadir los `using` necesarios en `ValidadorComunicacionAsincrona`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs`.
2. Inserta el bloque de importaciones inmediatamente antes del namespace; comprueba que el ancla aparece una sola vez.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `namespace AceriaApp.Api.ComunicacionAsincrona;`. Debe aparecer una sola vez.

```
using System.Collections.Generic;
using System.Linq;
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorComunicacionAsincrona.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Implementación funcional: `AceriaApp.Comunicacion.Infraestructura.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/AceriaApp.Comunicacion.Infraestructura.csproj`. Sustituye el contenido de la copia actual.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Comunicacion.Aplicacion/
AceriaApp.Comunicacion.Aplicacion.csproj" />
    <ProjectReference Include="../AceriaApp.Produccion.Infraestructura/
AceriaApp.Produccion.Infraestructura.csproj" />
    <PackageReference Include="Microsoft.Extensions.Http" Version="8.0.1" />
<PackageReference Include="RabbitMQ.Client" Version="7.1.2" /></ItemGroup>
</Project>
```

Declara una dependencia de compilación explícita; el ensamblado debe existir en la solución. Fija la versión de la dependencia para reproducir la compilación y sus API.

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add
src\AceriaApp.Comunicacion.Infraestructura\AceriaApp.Comunicacion.Infraestructura.csproj
```

Implementación funcional: `PlanificadorIntegrado.cs`

Aquí se une la transición del agregado de M5 con una intención de publicación. Ambos cambios pertenecen al mismo DbContext y al mismo SaveChanges. El despachador realiza después la operación externa; ante un fallo conserva el pendiente.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/PlanificadorIntegrado.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Text.Json;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Produccion.Dominio;
using AceriaApp.Produccion.Infraestructura;
using Microsoft.EntityFrameworkCore;
using Microsoft.Data.SqlClient;
namespace AceriaApp.Comunicacion.Infraestructura;
public sealed class PlanificadorIntegrado(ProduccionDbContext db) : IPlanificadorIntegrado
{
    public async Task<IReadOnlyList<LineaPlanificadaV1>?> ObtenerLineasAsync(Guid id, CancellationToken ct)
    {
        var orden = await db.Ordenes.AsNoTracking().Include(x => x.Lineas).SingleOrDefaultAsync(x => x.Id ==
id, ct);
        return orden?.Lineas.Select(x => new LineaPlanificadaV1(x.Material.Valor,
x.Cantidad.Valor)).ToArray();
    }
    public async Task<EstadoPlanificacion> PlanificarAsync(Guid id, CancellationToken ct)
    {
```

```
        var orden = await db.Ordenes.Include(x => x.Lineas).SingleOrDefaultAsync(x => x.Id == id, ct);
        if (orden is null) return EstadoPlanificacion.NoExiste;
        if (orden.Estado != EstadoOrdenProduccion.Borrador) return EstadoPlanificacion.Conflicto;
        orden.Planificar();
        var evento = new OrdenPlanificadaV1(Guid.NewGuid(), orden.Id, DateTimeOffset.UtcNow,
            orden.Lineas.Select(x => new LineaPlanificadaV1(x.Material.Valor, x.Cantidad.Valor)).ToArray());
        evento.Validar();
        db.MensajesSalida.Add(new MensajeSalida { Id = evento.EventoId, OrdenId = orden.Id,
            Contenido = JsonSerializer.Serialize(evento), CreadoEnUtc = evento.OcurridoEnUtc });
        try { await db.SaveChangesAsync(ct); }
        catch (DbUpdateConcurrencyException) { db.ChangeTracker.Clear(); return
EstadoPlanificacion.Conflicto; }
        catch (DbUpdateException ex) when (ex.InnerException is SqlException sql && (sql.Number == 2601 ||
sql.Number == 2627))
        { db.ChangeTracker.Clear(); return EstadoPlanificacion.Conflicto; }
        return EstadoPlanificacion.Aceptada;
    }
}
public sealed class DespachadorOutbox(ProduccionDbContext db, ITransporteEventos transporte)
{
    public async Task<int> DespacharAsync(CancellationToken ct)
    {
        var pendientes = await db.MensajesSalida.Where(x => x.EnviadoEnUtc == null)
            .OrderBy(x => x.CreadoEnUtc).ThenBy(x => x.Id).Take(20).ToListAsync(ct);
        foreach (var mensaje in pendientes)
        {
            await transporte.PublicarAsync(mensaje.Contenido, ct);
            mensaje.EnviadoEnUtc = DateTimeOffset.UtcNow;
            await db.SaveChangesAsync(ct);
        }
        return pendientes.Count;
    }
}
```

Obtiene datos para lectura sin incorporarlos al seguimiento de cambios. Confirma juntos el cambio de estado de la orden y el outbox en la transacción local de SQL; el broker participa después. Reconoce la carrera en la restricción única de SQL sin ocultar otros errores de persistencia.

Distingue una salida pendiente de una confirmada por el broker; el marcado posterior puede repetirse tras un fallo. Persiste el marcado del outbox después de recibir confirmación del broker; un fallo antes de este guardado puede causar una nueva publicación.

Implementación funcional: `ConflictoProduccionException.cs`

Define el conflicto de concurrencia en aplicación sin referenciar EF. Infraestructura lo produce tras una escritura rechazada y el caso de uso lo transforma en el resultado que la API devuelve como 409.

Archivo completo: `src/AceriaApp.Produccion.Aplicacion/ConflictoProduccionException.cs`. Crea el archivo y sus carpetas si no existen.

```
namespace AceriaApp.Produccion.Aplicacion;
public sealed class ConflictoProduccionException() : Exception("La orden cambió desde su lectura; vuelve a
consultarla.");
```

Expresa el conflicto de concurrencia sin exponer una excepción de EF a aplicación; el caso de uso lo traduce al resultado que la API devuelve como 409.

Implementación funcional: `OrdenesProduccionCasosUso.cs`

Los casos de uso mantienen las reglas del agregado y traducen ConflictoProduccionException al error orden.concurrente de tipo Conflicto. El adaptador HTTP devuelve 409 para que el cliente vuelva a consultar antes de decidir otro intento.

Archivo completo: `src/AceriaApp.Produccion.Aplicacion/OrdenesProduccionCasosUso.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Produccion.Dominio;
namespace AceriaApp.Produccion.Aplicacion;

// La aplicación coordina; las reglas de capacidad y de estado viven en el agregado.
public sealed class OrdenesProduccionCasosUso(
    IOrdenProduccionRepositorio repositorio, IProduccionUnitOfWork unidad)
{
    public async Task<ResultadoProduccion<Guid>> CrearAsync(string cliente, string material, int cantidad,
CancellationToken ct = default)
    {
        try
        {
            var orden = new OrdenProduccion(cliente, material, cantidad);
            await repositorio.AgregarAsync(orden, ct);
            await unidad.CommitAsync(ct);
            return ResultadoProduccion<Guid>.Exito(orden.Id);
        }
        catch (ReglaProduccionException ex) { return ResultadoProduccion<Guid>.Fallo(Traducir(ex)); }
    }
    public async Task<ResultadoProduccion<OrdenProduccionDto>> ObtenerAsync(Guid id, CancellationToken ct =
default)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        return orden is null ? Ausente() :
ResultadoProduccion<OrdenProduccionDto>.Exito(OrdenProduccionDto.Desde(orden));
    }
    public Task<ResultadoProduccion<OrdenProduccionDto>> PlanificarAsync(Guid id, CancellationToken ct =
default) =>
        ModificarAsync(id, o => o.Planificar(), ct);
    public Task<ResultadoProduccion<OrdenProduccionDto>> FinalizarAsync(Guid id, CancellationToken ct =
default) =>
        ModificarAsync(id, o => o.Finalizar(), ct);

    public Task<ResultadoProduccion<OrdenProduccionDto>> AgregarLineaAsync(Guid id, string material, int
cantidad, CancellationToken ct = default) =>
        ModificarAsync(id, o => o.AgregarLinea(material, cantidad), ct);

    private async Task<ResultadoProduccion<OrdenProduccionDto>> ModificarAsync(Guid id,
Action<OrdenProduccion> cambio, CancellationToken ct)
    {
        var orden = await repositorio.ObtenerAsync(id, ct);
        if (orden is null) return Ausente();
        try
        {
            cambio(orden);
            await unidad.CommitAsync(ct);
            return ResultadoProduccion<OrdenProduccionDto>.Exito(OrdenProduccionDto.Desde(orden));
        }
        catch (ReglaProduccionException ex) { return
ResultadoProduccion<OrdenProduccionDto>.Fallo(Traducir(ex)); }
        catch (ConflictoProduccionException ex)
        { return ResultadoProduccion<OrdenProduccionDto>.Fallo(new("orden.concurrente", ex.Message,
TipoErrorProduccion.Conflicto)); }
    }
    private static ResultadoProduccion<OrdenProduccionDto> Ausente() =>
        ResultadoProduccion<OrdenProduccionDto>.Fallo(new("orden.ausente", "La orden de producción no
existe.", TipoErrorProduccion.NoEncontrado));
    private static ErrorProduccion Traducir(ReglaProduccionException ex) => new(ex.Codigo, ex.Message,
        ex.Codigo == "estado.invalido" ? TipoErrorProduccion.Conflicto : TipoErrorProduccion.Validacion);
}
```

Implementación funcional: `OrdenProduccion.cs`

La copia del agregado incorpora Revision y la incrementa al modificar líneas o cambiar de estado. Este contador pertenece a la raíz y permite detectar que otra operación cambió el agregado desde su lectura; la protección incluye cambios que mantienen el estado Borrador.

Archivo completo: `src/AceriaApp.Produccion.Dominio/OrdenProduccion.cs`. Sustituye el contenido de la copia actual.

```
namespace AceriaApp.Produccion.Dominio;
```

```
public sealed class OrdenProduccion : IEquatable<OrdenProduccion>
{
    public const int CapacidadMaxima = 500;
    private readonly List<LineaDeOrden> _lineas = new();
    public Guid Id { get; private set; }
    public long Revision { get; private set; }
    public string Cliente { get; private set; } = "";
    public EstadoOrdenProduccion Estado { get; private set; }
    public IReadOnlyList<LineaDeOrden> Lineas => _lineas.AsReadOnly();
    public int CantidadTotal => _lineas.Sum(x => x.Cantidad.Valor);
    // Compatibilidad con el primer ejemplo de 5.1; la raíz calcula el total.
    public int Cantidad => CantidadTotal;
    public string Material => _lineas[0].Material.Valor;

    private OrdenProduccion() { }
    public OrdenProduccion(string cliente, string material, int cantidad)
    {
        if (string.IsNullOrWhiteSpace(cliente) || cliente.Trim().Length > 100)
            throw new ReglaProduccionException("cliente.invalido", "El cliente debe tener entre 1 y 100
caracteres.");
        Id = Guid.NewGuid();
        Cliente = cliente.Trim();
        Estado = EstadoOrdenProduccion.Borrador;
        AgregarLinea(material, cantidad);
    }

    public void AgregarLinea(string material, int cantidad)
    {
        ExigirBorrador();
        var nombre = new NombreMaterial(material);
        var valor = new CantidadProduccion(cantidad);
        ExigirCapacidad(CantidadTotal + valor.Valor);
        _lineas.Add(new LineaDeOrden(nombre, valor));
        Revision++;
    }

    public void CambiarCantidadLinea(Guid idLinea, int cantidad)
    {
        ExigirBorrador();
        var linea = _lineas.SingleOrDefault(x => x.Id == idLinea)
            ?? throw new ReglaProduccionException("linea.inexistente", "La línea no pertenece a la orden.");
        var nueva = new CantidadProduccion(cantidad);
        ExigirCapacidad(CantidadTotal - linea.Cantidad.Valor + nueva.Valor);
        linea.CambiarCantidad(nueva);
        Revision++;
    }

    public void Planificar()
    {
        ExigirBorrador();
        Estado = EstadoOrdenProduccion.Planificada;
        Revision++;
    }
    public void Finalizar()
    {
        if (Estado != EstadoOrdenProduccion.Planificada)
            throw new ReglaProduccionException("estado.invalido", "Solo se puede finalizar una orden
planificada.");
        Estado = EstadoOrdenProduccion.Finalizada;
        Revision++;
    }
    private void ExigirBorrador()
    {
        if (Estado != EstadoOrdenProduccion.Borrador)
            throw new ReglaProduccionException("estado.invalido", "La operación requiere una orden en
borrador.");
    }
    private static void ExigirCapacidad(int total)
    {
        if (total > CapacidadMaxima)
            throw new ReglaProduccionException("capacidad.excedida", "El total de la orden no puede superar
500 unidades.");
    }
    public bool Equals(OrdenProduccion? otra) => otra is not null && Id == otra.Id;
    public override bool Equals(object? otra) => otra is OrdenProduccion orden && Equals(orden);
```

```
    public override int GetHashCode() => Id.GetHashCode();
}
```

Incrementa la versión de la raíz después de una mutación válida; así SQL puede detectar cambios concurrentes aunque el estado siga siendo Borrador.

Implementación funcional: `MensajeSalida.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Produccion.Infraestructura/MensajeSalida.cs`. Crea el archivo y sus carpetas si no existen.

```
namespace AceriaApp.Produccion.Infraestructura;
public sealed class MensajeSalida
{
    public Guid Id { get; set; }
    public Guid OrdenId { get; set; }
    public string Contenido { get; set; } = "";
    public DateTimeOffset CreadoEnUtc { get; set; }
    public DateTimeOffset? EnviadoEnUtc { get; set; }
}
```

Implementación funcional: `PersistenciaProduccion.cs`

La unidad de trabajo convierte el conflicto optimista de EF en ConflictoProduccionException. Limpia el seguimiento de la escritura rechazada y deja que aplicación comunique el conflicto sin depender de tipos de EF.

Archivo completo: `src/AceriaApp.Produccion.Infraestructura/PersistenciaProduccion.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Produccion.Dominio;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Produccion.Infraestructura;
public sealed class OrdenProduccionRepositorio(ProduccionDbContext db) : IOrdenProduccionRepositorio
{
    public Task<OrdenProduccion?> ObtenerAsync(Guid id, CancellationToken ct = default) =>
        db.Ordenes.Include(o => o.Lineas).SingleOrDefaultAsync(o => o.Id == id, ct);
    public async Task AgregarAsync(OrdenProduccion orden, CancellationToken ct = default) =>
        await db.Ordenes.AddAsync(orden, ct);
}
public sealed class ProduccionUnitOfWork(ProduccionDbContext db) : IProduccionUnitOfWork
{
    public async Task CommitAsync(CancellationToken ct = default)
    {
        try { await db.SaveChangesAsync(ct); }
        catch (DbUpdateConcurrencyException)
        {
            db.ChangeTracker.Clear();
            throw new ConflictoProduccionException();
        }
    }
}
```

Confirma los cambios pendientes en la transacción local de SQL; esta operación no incluye al broker.

Implementación funcional: `ProduccionDbContext.cs`

Se amplía únicamente la copia de M6 con outbox y control de concurrencia por Revision. EF compara la versión original de la raíz en cada escritura; las mutaciones de líneas y estado la incrementan. Los archivos originales de M5 permanecen intactos. Las bases de M6 tienen nombres propios para que EnsureCreated no se confunda con una migración.

Archivo completo: `src/AceriaApp.Produccion.Infraestructura/ProduccionDbContext.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Produccion.Dominio;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Produccion.Infraestructura;
public sealed class ProduccionDbContext(DbContextOptions<ProduccionDbContext> opciones) : DbContext(opciones)
{
    public DbSet<OrdenProduccion> Ordenes => Set<OrdenProduccion>();
    public DbSet<MensajeSalida> MensajesSalida => Set<MensajeSalida>();
    protected override void OnModelCreating(ModelBuilder model)
    {
        var salida = model.Entity<MensajeSalida>();
        salida.ToTable("MensajesSalida");
        salida.HasKey(x => x.Id);
        salida.HasIndex(x => x.OrdenId).IsUnique();
        salida.Property(x => x.Contenido).IsRequired();
        var orden = model.Entity<OrdenProduccion>();
        orden.Property(x => x.Revision).IsConcurrencyToken();
        orden.ToTable("OrdenesProduccion");
        orden.HasKey(o => o.Id);
        orden.Property(o => o.Id).ValueGeneratedNever();
        orden.Property(o => o.Cliente).HasMaxLength(100).IsRequired();
        orden.Property(o => o.Estado).HasConversion<string>().HasMaxLength(20);
        orden.Ignore(o => o.CantidadTotal);
        orden.Ignore(o => o.Cantidad);
        orden.Ignore(o => o.Material);
        orden.HasMany(o =>
o.Lineas).WithOne().HasForeignKey("OrdenId").IsRequired().OnDelete(DeleteBehavior.Cascade);
        orden.Navigation(o => o.Lineas).HasField("_lineas").UsePropertyAccessMode(PropertyAccessMode.Field);
        var linea = model.Entity<LineaDeOrden>();
        linea.ToTable("LineasProduccion");
        linea.HasKey(l => l.Id);
        linea.Property(l => l.Id).ValueGeneratedNever();
        linea.Property(l => l.Material).HasConversion(v => v.Valor, v => new
NombreMaterial(v)).HasMaxLength(100).IsRequired();
        linea.Property(l => l.Cantidad).HasConversion(v => v.Valor, v => new CantidadProduccion(v));
    }
}
```

La restricción única de SQL protege la identidad también cuando dos operaciones concurrentes compiten. EF compara la Revision original de la raíz al actualizar. Toda mutación, tanto de líneas como de estado, incrementa esa versión para rechazar escrituras basadas en una lectura antigua.

PASO 6.2.9 — Crear la clase `CatalogoEventos`

Un catálogo en memoria sirve para comparar ejemplos, pero no es una cola AMQP. Los mensajes y las reentregas se observan con RabbitMQ y los cambios de Inventario.

1. Crea `src/AceriaApp.Api/ComunicacionAsincrona/CatalogoEventos.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/CatalogoEventos.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.ComunicacionAsincrona;

public class CatalogoEventos
{
    public static List<Evento> EventosPorDefecto()
    {
        return new List<Evento>
        {
            new() { Nombre = "OrdenCreada", Tipo = "Integracion", Publicador = "Ordenes.Api",
NumeroDeConsumidores = 2, TieneIdempotencia = true, TieneReintentos = true },
            new() { Nombre = "OrdenCancelada", Tipo = "Integracion", Publicador = "Ordenes.Api",
```

```
NumeroDeConsumidores = 2, TieneIdempotencia = true, TieneReintentos = true },
            new() { Nombre = "MaterialReservado", Tipo = "Integracion", Publicador = "Inventario.Api",
NumeroDeConsumidores = 1, TieneIdempotencia = true, TieneReintentos = true },
            new() { Nombre = "FacturaEmitida", Tipo = "Integracion", Publicador = "Facturacion.Api",
NumeroDeConsumidores = 1, TieneIdempotencia = true, TieneReintentos = true }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Construye una entrada del catálogo o del caso de prueba con valores explícitos. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoEventos` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.ComunicacionAsincrona` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

Implementación funcional: `PublicadorOutboxWorker.cs`

Un ámbito nuevo por iteración mantiene aislado el DbContext. Los fallos se registran y se reintentan con pausa; el bucle no afirma que haya entrega si el broker no confirma. El laboratorio no incorpora un límite automático de antigüedad de pendientes.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/PublicadorOutboxWorker.cs`. Crea el archivo y sus carpetas si no existen.

```
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;
namespace AceriaApp.Comunicacion.Infraestructura;
public sealed class PublicadorOutboxWorker(IServiceScopeFactory scopes, ILogger<PublicadorOutboxWorker>
log) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            try
            {
                using var scope = scopes.CreateScope();
                await scope.ServiceProvider.GetRequiredService<DespachadorOutbox>().DespacharAsync(ct);
            }
            catch (OperationCanceledException) when (ct.IsCancellationRequested) { break; }
            catch (Exception ex) { log.LogWarning(ex, "Outbox conserva los pendientes; se reintentará."); }
            try { await Task.Delay(TimeSpan.FromSeconds(2), ct); }
            catch (OperationCanceledException) when (ct.IsCancellationRequested) { break; }
        }
    }
}
```

Crea un ámbito por iteración o mensaje; el DbContext no se comparte entre trabajos concurrentes.

Implementación funcional: `RabbitMqTransporte.cs`

El adaptador usa RabbitMQ.Client 7.1.2 y AMQP real. Una conexión se reutiliza y cada operación tiene su canal. Las confirmaciones del publicador, el ACK del consumidor y el commit SQL son tres hechos distintos. BasicGet permite observar cada entrega en el laboratorio.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/RabbitMqTransporte.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Text;
using System.Text.Json;
using AceriaApp.Comunicacion.Aplicacion;
using RabbitMQ.Client;
namespace AceriaApp.Comunicacion.Infraestructura;
public sealed record RabbitMqOpciones(string Uri, string Prefijo = "aceria");
public sealed class RabbitMqTransporte(RabbitMqOpciones opciones) : ITransporteEventos, IAsyncDisposable
{
    private IConnection? conexion;
    private readonly SemaphoreSlim acceso = new(1, 1);
    private string Exchange => opciones.Prefijo + ".eventos";
    private string Cola => opciones.Prefijo + ".inventario.planificada.v1";
    private const string Ruta = "orden.planificada.v1";
    private async Task<IChannel> CanalAsync(bool confirmar, CancellationToken ct)
    {
        await acceso.WaitAsync(ct);
        try
        {
            if (conexion is null || !conexion.IsOpen)
            {
                if (conexion is not null) await conexion.DisposeAsync();
                conexion = await new ConnectionFactory { Uri = new Uri(opciones.Uri),
                    AutomaticRecoveryEnabled = true, RequestedConnectionTimeout = TimeSpan.FromSeconds(5) }
                    .CreateConnectionAsync(ct);
            }
            var canal = await conexion.CreateChannelAsync(new CreateChannelOptions(confirmar, confirmar),
ct);
            await canal.ExchangeDeclareAsync(Exchange, ExchangeType.Direct, durable: true, autoDelete: false,
cancellationToken: ct);
            await canal.QueueDeclareAsync(Cola + ".errores", durable: true, exclusive: false, autoDelete:
false, cancellationToken: ct);
            await canal.QueueDeclareAsync(Cola, durable: true, exclusive: false, autoDelete: false,
                arguments: new Dictionary<string, object?> { ["x-dead-letter-exchange"] = "", ["x-dead-
letter-routing-key"] = Cola + ".errores" }, cancellationToken: ct);
            await canal.QueueBindAsync(Cola, Exchange, Ruta, cancellationToken: ct);
            return canal;
        }
        finally { acceso.Release(); }
    }
    public async Task PublicarAsync(string contenido, CancellationToken ct)
    {
        using var plazo = CancellationTokenSource.CreateLinkedTokenSource(ct);
        plazo.CancelAfter(TimeSpan.FromSeconds(10));
        await using var canal = await CanalAsync(true, plazo.Token);
        await canal.BasicPublishAsync(Exchange, Ruta, mandatory: true,
            basicProperties: new BasicProperties { Persistent = true, ContentType = "application/json", Type
= "orden.planificada.v1" },
            body: Encoding.UTF8.GetBytes(contenido), cancellationToken: plazo.Token);
    }
    public async Task<bool> ConsumirUnoAsync(Func<string, CancellationToken, Task> procesar,
CancellationToken ct)
    {
        await using var canal = await CanalAsync(false, ct);
        var entrega = await canal.BasicGetAsync(Cola, autoAck: false, cancellationToken: ct);
        if (entrega is null) return false;
        try
        {
            await procesar(Encoding.UTF8.GetString(entrega.Body.Span), ct);
            await canal.BasicAckAsync(entrega.DeliveryTag, multiple: false, cancellationToken: ct);
        }
        catch (Exception ex) when (ex is JsonException or InvalidDataException)
        { await canal.BasicNackAsync(entrega.DeliveryTag, multiple: false, requeue: false, cancellationToken:
ct); }
        catch
        {
            // Si el canal ya se cerró, RabbitMQ vuelve a poner en cola la entrega sin confirmar.
            if (canal.IsOpen) await canal.BasicNackAsync(entrega.DeliveryTag, multiple: false, requeue: true,
cancellationToken: CancellationToken.None);
            throw;
        }
        return true;
    }
    public async Task<uint> ContarErroresAsync(CancellationToken ct)
    {
        await using var canal = await CanalAsync(false, ct);
```

```
        return (await canal.QueueDeclarePassiveAsync(Cola + ".errores", ct)).MessageCount;
    }
    public async Task EliminarTopologiaAsync(CancellationToken ct)
    {
        await using var canal = await CanalAsync(false, ct);
        await canal.QueueDeleteAsync(Cola, false, false, ct);
        await canal.QueueDeleteAsync(Cola + ".errores", false, false, ct);
        await canal.ExchangeDeleteAsync(Exchange, cancellationToken: ct);
    }
    public async ValueTask DisposeAsync()
    {
        if (conexion is not null) await conexion.DisposeAsync();
        acceso.Dispose();
    }
}
```

Dispone la conexión existente antes de reemplazarla o de liberar definitivamente el adaptador. Configura confirms y seguimiento según confirmar: la publicación usa true y el consumo usa false. Declara la topología durable usada por el publicador y el consumidor.

Combina la cancelación del solicitante con un límite propio de la operación remota. Acota la espera de confirmación; un timeout puede dejar incierto si el broker recibió el mensaje. Publica con confirmaciones activadas: esperar acredita aceptación del broker, no ejecución del consumidor.

Extrae una entrega sin confirmación automática; es polling didáctico, no suscripción push.

PASO 6.2.10 — Añadir el `using` necesario en `CatalogoEventos`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/ComunicacionAsincrona/CatalogoEventos.cs`.
2. Inserta el bloque de importaciones inmediatamente antes del namespace; comprueba que el ancla aparece una sola vez.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/CatalogoEventos.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `namespace AceriaApp.Api.ComunicacionAsincrona;`. Debe aparecer una sola vez.

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoEventos.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Implementación funcional: `AceriaApp.Inventario.Api.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj`. Sustituye el contenido de la copia actual.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../AceriaApp.Comunicacion.Contratos/AceriaApp.Comunicacion.Contratos.csproj" /
>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.21" />
<ProjectReference Include="../AceriaApp.Comunicacion.Infraestructura/
AceriaApp.Comunicacion.Infraestructura.csproj" /></ItemGroup>
</Project>
```

Desde la carpeta de esta etapa, registra el proyecto:

```
dotnet sln AceriaApp.sln add src\AceriaApp.Inventario.Api\AceriaApp.Inventario.Api.csproj
```

Implementación funcional: `ConsumidorPlanificacionWorker.cs`

Convierte el JSON recibido en un contrato, abre un ámbito SQL y delega el efecto durable. El transporte confirma después. La pausa evita un bucle ocupado; el polling se usa para entender el flujo, no como recomendación de máximo rendimiento.

Archivo completo: `src/AceriaApp.Inventario.Api/ConsumidorPlanificacionWorker.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Text.Json;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
namespace AceriaApp.Inventario.Api;
public sealed class ConsumidorPlanificacionWorker(RabbitMqTransporte bus, IServiceScopeFactory scopes,
    ILogger<ConsumidorPlanificacionWorker> log) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            try
            {
                await bus.ConsumirUnoAsync(async (json, token) =>
                {
                    var evento = JsonSerializer.Deserialize<OrdenPlanificadaV1>(json)
                        ?? throw new InvalidDataException("Evento nulo.");
                    using var scope = scopes.CreateScope();
                    await
scope.ServiceProvider.GetRequiredService<ReceptorPlanificacion>().ProcesarAsync(evento, token);
                }, ct);
            }
            catch (OperationCanceledException) when (ct.IsCancellationRequested) { break; }
            catch (Exception ex) { log.LogWarning(ex, "La entrega no se confirma; se reintentará."); }
            try { await Task.Delay(TimeSpan.FromMilliseconds(250), ct); }
            catch (OperationCanceledException) when (ct.IsCancellationRequested) { break; }
        }
    }
}
```

Implementación funcional: `InventarioDbContext.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/InventarioDbContext.cs`. Sustituye el contenido de la copia actual.

```
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class Existencia
{
```

```
    public string Material { get; set; } = "";
    public int Stock { get; set; }
}
public sealed class InventarioDbContext(DbContextOptions<InventarioDbContext> opciones) : DbContext(opciones)
{
    public DbSet<Existencia> Existencias => Set<Existencia>();
    public DbSet<PlanificacionRecibida> Planificaciones => Set<PlanificacionRecibida>();
    protected override void OnModelCreating(ModelBuilder model)
    {
        model.Entity<PlanificacionRecibida>().HasKey(x => x.EventoId);
        model.Entity<PlanificacionRecibida>().HasIndex(x => x.OrdenId).IsUnique();
        model.Entity<PlanificacionRecibida>().Property(x => x.HuellaContenido).HasMaxLength(64);
        model.Entity<Existencia>().HasKey(x => x.Material);
        model.Entity<Existencia>().Property(x => x.Material).HasMaxLength(100);
    }
}
```

Implementación funcional: `InventarioProgram.cs`

Este ejecutable es un segundo proceso ASP.NET Core con su propio puerto y base SQL. No lee directamente tablas de Producción. Inicializa existencias de laboratorio y expone consultas; cuando se habilita, aloja el consumidor de eventos.

Archivo completo: `src/AceriaApp.Inventario.Api/InventarioProgram.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class InventarioProgram
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddDbContext<InventarioDbContext>(o => o.UseSqlServer(
            builder.Configuration.GetConnectionString("AceriaInventario") ??
            @"Server=(localdb)
\MSSQLLocalDB;Database=AceriaInventarioM06;Trusted_Connection=True;TrustServerCertificate=True"));
        builder.Services.AddScoped<ReceptorPlanificacion>();
        builder.Services.AddSingleton(new
RabbitMqOpciones(builder.Configuration["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            builder.Configuration["Comunicacion:Prefijo"] ?? "aceria"));
        builder.Services.AddSingleton<RabbitMqTransporte>();
        if (builder.Configuration.GetValue<bool>("Comunicacion:WorkersActivos"))
            builder.Services.AddHostedService<ConsumidorPlanificacionWorker>();
        var app = builder.Build();
        if (app.Environment.IsDevelopment() && !
app.Configuration.GetValue<bool>("SkipDatabaseInitialization"))
        {
            using var scope = app.Services.CreateScope();
            var db = scope.ServiceProvider.GetRequiredService<InventarioDbContext>();
            await db.Database.EnsureCreatedAsync();
            if (!await db.Existencias.AnyAsync())
            {
                db.Existencias.AddRange(new Existencia { Material = "Acero", Stock = 300 },
                    new Existencia { Material = "Inoxidable", Stock = 120 });
                await db.SaveChangesAsync();
            }
        }
        app.MapGet("/health", () => Results.Ok(new { servicio = "Inventario" }));
        app.MapGet("/api/v1/materiales/{material}", async (string material, InventarioDbContext db,
CancellationToken ct) =>
        {
            var valor = await db.Existencias.AsNoTracking().SingleOrDefaultAsync(x => x.Material == material,
ct);
            return valor is null ? Results.NotFound() : Results.Ok(new MaterialDto(valor.Material,
valor.Stock));
        });
        app.MapGet("/api/v1/planificaciones/{ordenId:guid}", async (Guid ordenId, InventarioDbContext db,
CancellationToken ct) =>
        {
```

```
            var valor = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.OrdenId ==
ordenId, ct);
            return valor is null ? Results.NotFound() : Results.Ok(new { valor.EventoId, valor.OrdenId,
valor.CantidadTotal });
        });
        await app.RunAsync();
    }
}
```

Mantiene SQL Server real como proveedor; no se sustituye por almacenamiento en memoria. Habilita el proceso periódico únicamente cuando la configuración lo solicita. Inicializa una base didáctica nueva. No migra un esquema existente: usa los nombres separados de cada etapa.

Guarda las existencias iniciales del laboratorio en la base propia de Inventario. Publica una consulta HTTP con un contrato de respuesta verificable.

Implementación funcional: `PlanificacionRecibida.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/PlanificacionRecibida.cs`. Crea el archivo y sus carpetas si no existen.

```
namespace AceriaApp.Inventario.Api;
public sealed class PlanificacionRecibida
{
    public Guid EventoId { get; set; }
    public Guid OrdenId { get; set; }
    public int CantidadTotal { get; set; }
    public string HuellaContenido { get; set; } = "";
    public DateTimeOffset RecibidoEnUtc { get; set; }
}
```

Implementación funcional: `ReceptorPlanificacion.cs`

La proyección y la marca de recepción son la misma fila SQL. La clave del evento evita efectos repetidos; su huella rechaza la misma identidad con contenido distinto. El índice por orden evita registrar otra planificación para el mismo agregado.

Archivo completo: `src/AceriaApp.Inventario.Api/ReceptorPlanificacion.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Security.Cryptography;
using System.Text.Json;
using AceriaApp.Comunicacion.Contratos;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
namespace AceriaApp.Inventario.Api;
public sealed class ReceptorPlanificacion(InventarioDbContext db)
{
    public async Task<bool> ProcesarAsync(OrdenPlanificadaV1 evento, CancellationToken ct)
    {
        evento.Validar();
        var huella = Convert.ToHexString(SHA256.HashData(JsonSerializer.SerializeToUtf8Bytes(evento)));
        var previo = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.EventoId ==
evento.EventoId, ct);
        if (previo is not null) return Comparar(previo.HuellaContenido, huella);
        db.Planificaciones.Add(new PlanificacionRecibida { EventoId = evento.EventoId, OrdenId =
evento.OrdenId,
            CantidadTotal = evento.Lineas.Sum(x => x.Cantidad), HuellaContenido = huella, RecibidoEnUtc =
DateTimeOffset.UtcNow });
        try { await db.SaveChangesAsync(ct); return true; }
        catch (DbUpdateException ex) when (ex.InnerException is SqlException sql && (sql.Number == 2601 ||
sql.Number == 2627))
        {
            db.ChangeTracker.Clear();
```

```
            var guardado = await db.Planificaciones.AsNoTracking().SingleOrDefaultAsync(x => x.EventoId ==
evento.EventoId, ct);
            if (guardado is null) throw new InvalidDataException("La orden ya tiene otro evento de
planificación.", ex);
            return Comparar(guardado.HuellaContenido, huella);
        }
    }
    private static bool Comparar(string anterior, string nueva)
    {
        if (anterior != nueva) throw new InvalidDataException("EventoId reutilizado con contenido
diferente.");
        return false;
    }
}
```

Calcula una huella del contenido para distinguir un duplicado legítimo de una identidad reutilizada con otros datos. Guarda la fila que representa inbox y proyección en la transacción SQL local; el transporte confirma la entrega después de que termine.

PASO 6.2.11 — Crear el proyecto de tests del punto 6.2

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.ComunicacionAsincrona.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 14 y 15 incluyen las clases de prueba completas.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj` y `dotnet restore tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj`.
Operación: Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.
- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 6.2.12 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj` y `dotnet build tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

PASO 6.2.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj` y `dotnet list tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj package`.
4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj`.
Operación: Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
```

```
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

Implementación funcional: `ComunicacionEndpoints.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/Comunicacion/ComunicacionEndpoints.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Comunicacion.Aplicacion;
namespace AceriaApp.Api.Comunicacion;
public static class ComunicacionEndpoints
{
    public static void MapComunicacion(this WebApplication app)
    {
        app.MapGet("/api/v1/comunicacion/materiales/{material}", async (string material, IClienteInventario
inventario, CancellationToken ct) =>
        {
            try
            {
                var dto = await inventario.ObtenerAsync(material, ct);
                return dto is null ? Results.NotFound() : Results.Ok(dto);
            }
            catch (InventarioNoDisponibleException)
            { return Results.Problem(statusCode: 503, title: "Inventario no está disponible."); }
        }).WithTags("Comunicacion");
        app.MapPost("/api/v1/comunicacion/ordenes/{id:guid}/planificar", async (Guid id,
IPlanificadorIntegrado planificador, CancellationToken ct) =>
        {
            var estado = await planificador.PlanificarAsync(id, ct);
            return estado switch
            {
                EstadoPlanificacion.NoExiste => Results.NotFound(),
                EstadoPlanificacion.Conflicto => Results.Problem(statusCode: 409, title: "La orden no está en
borrador."),
                _ => Results.Accepted($"/api/v1/produccion/ordenes/{id}", new { ordenId = id, estado =
"Planificada", propagacion = "Pendiente" })
            };
        }).WithTags("Comunicacion");
    }
}
```

Publica la operación de planificación; el adaptador traduce el resultado del caso de uso a HTTP. Devuelve 202: la planificación local está confirmada y la propagación a Inventario es eventual.

Implementación funcional: `appsettings.json`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/appsettings.json`. Sustituye el contenido de la copia actual.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM062;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM062;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM062;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m062",
    "WorkersActivos": false
  }
}
```

Separa los almacenes de esta etapa y conserva los de los módulos anteriores.

Implementación funcional: `RegistroComunicacion.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/RegistroComunicacion.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Comunicacion.Aplicacion;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
namespace AceriaApp.Comunicacion.Infraestructura;
public static class RegistroComunicacion
{
    public static IServiceCollection AddComunicacion(this IServiceCollection services, IConfiguration config)
    {
        services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>(http =>
        {
            http.BaseAddress = new Uri(config["Comunicacion:InventarioUrl"] ?? "http://localhost:5106/");
            http.Timeout = TimeSpan.FromSeconds(3);
        });
        services.AddScoped<IPlanificadorIntegrado, PlanificadorIntegrado>();
        services.AddScoped<DespachadorOutbox>();
        services.AddSingleton(new RabbitMqOpciones(config["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            config["Comunicacion:Prefijo"] ?? "aceria"));
        services.AddSingleton<RabbitMqTransporte>();
        services.AddSingleton<ITransporteEventos>(sp => sp.GetRequiredService<RabbitMqTransporte>());
        if (config.GetValue<bool>("Comunicacion:WorkersActivos"))
services.AddHostedService<PublicadorOutboxWorker>();
        return services;
    }
}
```

Registra el cliente tipado y centraliza dirección y timeout; los handlers se administran mediante la factoría.

Implementación funcional: `appsettings.json`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/appsettings.json`. Sustituye el contenido de la copia actual.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM062;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m062",
    "WorkersActivos": false
  }
}
```

PASO 6.2.14 — Crear el archivo `ValidadorComunicacionAsincronaTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.ComunicacionAsincrona.Tests/ValidadorComunicacionAsincronaTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.ComunicacionAsincrona.Tests/ValidadorComunicacionAsincronaTests.cs`.
Operación: Crear.

```
using AceriaApp.Api.ComunicacionAsincrona;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ComunicacionAsincrona.Tests;

public class ValidadorComunicacionAsincronaTests
{
    private readonly ValidadorComunicacionAsincrona _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoEventos.EventosPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinEventos_DevuelveError()
    {
        var resultado = _validador.Validar(new List<Evento>());

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay eventos"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
```

```
    {
        var eventos = new List<Evento>
        {
            new() { Nombre = "X", Tipo = "TipoInventado", Publicador = "A", NumeroDeConsumidores = 1,
TieneIdempotencia = true, TieneReintentos = true }
        };

        var resultado = _validador.Validar(eventos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Tipo no permitido"));
    }

    [Fact]
    public void Validar_SinPublicador_DevuelveError()
    {
        var eventos = new List<Evento>
        {
            new() { Nombre = "X", Tipo = "Integracion", Publicador = string.Empty, NumeroDeConsumidores = 1,
TieneIdempotencia = true, TieneReintentos = true }
        };

        var resultado = _validador.Validar(eventos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara el publicador"));
    }

    [Fact]
    public void Validar_SinConsumidores_DevuelveError()
    {
        var eventos = new List<Evento>
        {
            new() { Nombre = "X", Tipo = "Integracion", Publicador = "A", NumeroDeConsumidores = 0,
TieneIdempotencia = true, TieneReintentos = true }
        };

        var resultado = _validador.Validar(eventos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tiene consumidores"));
    }

    [Fact]
    public void Validar_SinIdempotencia_DevuelveError()
    {
        var eventos = new List<Evento>
        {
            new() { Nombre = "X", Tipo = "Integracion", Publicador = "A", NumeroDeConsumidores = 1,
TieneIdempotencia = false, TieneReintentos = true }
        };

        var resultado = _validador.Validar(eventos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara idempotencia"));
    }

    [Fact]
    public void Validar_SinReintentos_DevuelveError()
    {
        var eventos = new List<Evento>
        {
            new() { Nombre = "X", Tipo = "Integracion", Publicador = "A", NumeroDeConsumidores = 1,
TieneIdempotencia = true, TieneReintentos = false }
        };

        var resultado = _validador.Validar(eventos);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara reintentos"));
    }
    [Fact]
    public void Validar_EventoDominioLocalSinPoliticasDeEntrega_DevuelveValido()
    {
        var eventos = new List<Evento>
```

```
        {
            new() { Nombre = "OrdenPlanificada", Tipo = "Dominio", Publicador = "Produccion",
NumeroDeConsumidores = 0 }
        };

        var resultado = _validador.Validar(eventos);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Verificación concreta. `ValidadorComunicacionAsincronaTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 6.2.15 — Crear el archivo `CatalogoEventosTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.ComunicacionAsincrona.Tests/CatalogoEventosTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.ComunicacionAsincrona.Tests/CatalogoEventosTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.ComunicacionAsincrona;
using FluentAssertions;
using Xunit;

namespace AceriaApp.ComunicacionAsincrona.Tests;

public class CatalogoEventosTests
{
    [Fact]
    public void EventosPorDefecto_ContieneCuatroEventos()
    {
        var eventos = CatalogoEventos.EventosPorDefecto();

        eventos.Should().HaveCount(4);
    }

    [Fact]
    public void EventosPorDefecto_TodosDeIntegracion()
    {
        var eventos = CatalogoEventos.EventosPorDefecto();

        eventos.Should().OnlyContain(e => e.Tipo == "Integracion");
    }
```

```
    [Fact]
    public void EventosPorDefecto_TodosTienenIdempotencia()
    {
        var eventos = CatalogoEventos.EventosPorDefecto();

        eventos.Should().OnlyContain(e => e.TieneIdempotencia);
    }

    [Fact]
    public void EventosPorDefecto_TodosTienenReintentos()
    {
        var eventos = CatalogoEventos.EventosPorDefecto();

        eventos.Should().OnlyContain(e => e.TieneReintentos);
    }

    [Fact]
    public void EventosPorDefecto_TodosTienenConsumidores()
    {
        var eventos = CatalogoEventos.EventosPorDefecto();

        eventos.Should().OnlyContain(e => e.NumeroDeConsumidores >= 1);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Comprueba la condición en cada entrada declarada del catálogo.

Verificación concreta. `CatalogoEventosTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Implementación funcional: `AceriaApp.ComunicacionAsincrona.Tests.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj`. Sustituye el contenido de la copia actual.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
    <PackageReference Include="Testcontainers.RabbitMq" Version="3.10.0" />
</ItemGroup>
<ItemGroup><Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" /><Compile
Include="../Compartido/RabbitPruebas.cs" Link="RabbitPruebas.cs" /></ItemGroup>
</Project>
```

Implementación funcional: `ContratoEventoTests.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `tests/AceriaApp.ComunicacionAsincrona.Tests/ContratoEventoTests.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Contratos;
using Xunit;
namespace AceriaApp.ComunicacionAsincrona.Tests;
public class ContratoEventoTests
{
    private static OrdenPlanificadaV1 Valido() => new(Guid.NewGuid(), Guid.NewGuid(), DateTimeOffset.UtcNow,
new[] { new LineaPlanificadaV1("Acero", 20) });
    [Fact, Trait("Tipo","Unidad")]
    public void Evento_valido_preserva_identidad_y_datos() { var e=Valido(); e.Validar();
Assert.Equal(20,e.Lineas.Sum(x=>x.Cantidad)); }
    [Fact, Trait("Tipo","Unidad")]
    public void Version_desconocida_se_rechaza() { Assert.Throws<InvalidDataException>(()=>(Valido() with
{ Version=2 }).Validar()); }
    [Fact, Trait("Tipo","Unidad")]
    public void Identificadores_vacios_se_rechazan() { Assert.Throws<InvalidDataException>(()=>(Valido() with
{ EventoId=Guid.Empty }).Validar()); Assert.Throws<InvalidDataException>(()=>(Valido() with
{ OrdenId=Guid.Empty }).Validar()); }
    [Fact, Trait("Tipo","Unidad")]
    public void Cantidades_y_lineas_invalidas_se_rechazan()
{ Assert.Throws<InvalidDataException>(()=>(Valido() with
{ Lineas=Array.Empty<LineaPlanificadaV1>() }).Validar()); Assert.Throws<InvalidDataException>(()=>(Valido()
with { Lineas=new[]{new LineaPlanificadaV1("Acero",501)} }).Validar()); }
}
```

Comprueba una consecuencia observable. Si cambia el comportamiento descrito por el nombre del test, esta aserción debe fallar.

Implementación funcional: `OutboxInboxTests.cs`

Estas pruebas abren bases SQL Server reales y aisladas. Verifican atomicidad con un fallo SQL inducido, lecturas desde contextos nuevos, duplicados, carreras entre cambios de líneas y planificación, y pendientes durante un fallo del broker. TransportePrueba es un doble controlado para provocar fallos; RabbitMqTests valida el transporte real por separado.

Archivo completo: `tests/AceriaApp.ComunicacionAsincrona.Tests/OutboxInboxTests.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Text.Json;
using AceriaApp.Produccion.Aplicacion;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Inventario.Api;
using AceriaApp.Produccion.Dominio;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.PruebasSqlServer;
using Microsoft.EntityFrameworkCore;
using Xunit;
namespace AceriaApp.ComunicacionAsincrona.Tests;
public class OutboxInboxTests
{
    private static ProduccionDbContext Produccion(BaseSqlServer bd) => new(new
DbContextOptionsBuilder<ProduccionDbContext>().UseSqlServer(bd.ConnectionString).Options);
    private static InventarioDbContext Inventario(BaseSqlServer bd) => new(new
DbContextOptionsBuilder<InventarioDbContext>().UseSqlServer(bd.ConnectionString).Options);
    private static OrdenPlanificadaV1 Evento() =>
new(Guid.NewGuid(),Guid.NewGuid(),DateTimeOffset.UtcNow,new[]{new LineaPlanificadaV1("Acero",20)});
    [Fact]
    public async Task Planificar_guarda_estado_y_outbox_en_SQLServer()
    {
```

```
        await using var bd=BaseSqlServer.Crear(); await using var db=Produccion(bd); await
db.Database.EnsureCreatedAsync();
        var o=new OrdenProduccion("Axia","Acero",20);db.Ordenes.Add(o);await db.SaveChangesAsync();
        var r=await new
PlanificadorIntegrado(db).PlanificarAsync(o.Id,default);Assert.Equal(EstadoPlanificacion.Aceptada,r);
        await using var leer=Produccion(bd);Assert.Equal(EstadoOrdenProduccion.Planificada,(await
leer.Ordenes.SingleAsync()).Estado);
        Assert.Single(await
leer.MensajesSalida.ToListAsync());Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer",leer.Database.Provi
derName);
    }
    [Fact]
    public async Task Segunda_planificacion_no_crea_otro_evento()
    {
        await using var bd=BaseSqlServer.Crear();await using var db=Produccion(bd);await
db.Database.EnsureCreatedAsync();
        var o=new OrdenProduccion("Axia","Acero",20);db.Ordenes.Add(o);await db.SaveChangesAsync();var p=new
PlanificadorIntegrado(db);
        await p.PlanificarAsync(o.Id,default);Assert.Equal(EstadoPlanificacion.Conflicto,await
p.PlanificarAsync(o.Id,default));Assert.Equal(1,await db.MensajesSalida.CountAsync());
    }
    [Fact]
    public async Task Fallo_insertar_outbox_revierte_estado_de_orden()
    {
        await using var bd=BaseSqlServer.Crear();await using var db=Produccion(bd);await
db.Database.EnsureCreatedAsync();
        var o=new OrdenProduccion("Axia","Acero",20);db.Ordenes.Add(o);await db.SaveChangesAsync();
        await db.Database.ExecuteSqlRawAsync("CREATE TRIGGER BloquearSalida ON MensajesSalida AFTER INSERT AS
THROW 51001, 'Fallo inducido de outbox', 1;");
        await Assert.ThrowsAsync<DbUpdateException>(()=>new
PlanificadorIntegrado(db).PlanificarAsync(o.Id,default));
        await using var leer=Produccion(bd);Assert.Equal(EstadoOrdenProduccion.Borrador,(await
leer.Ordenes.SingleAsync()).Estado);Assert.Equal(0,await leer.MensajesSalida.CountAsync());
    }
    [Fact]
    public async Task Inbox_duplicado_conserva_un_unico_registro_persistido()
    {
        await using var bd=BaseSqlServer.Crear();await using var db=Inventario(bd);await
db.Database.EnsureCreatedAsync();var e=Evento();
        Assert.True(await new ReceptorPlanificacion(db).ProcesarAsync(e,default));
        await using var otro=Inventario(bd);Assert.False(await new
ReceptorPlanificacion(otro).ProcesarAsync(e,default));
        Assert.Equal(1,await otro.Planificaciones.CountAsync());Assert.Equal(20,(await
otro.Planificaciones.SingleAsync()).CantidadTotal);
    }
    [Fact]
    public async Task Misma_identidad_con_contenido_distinto_se_rechaza()
    {
        await using var bd=BaseSqlServer.Crear();await using var db=Inventario(bd);await
db.Database.EnsureCreatedAsync();var e=Evento();
        await new ReceptorPlanificacion(db).ProcesarAsync(e,default);
        await Assert.ThrowsAsync<InvalidDataException>(()=>new ReceptorPlanificacion(db).ProcesarAsync(e with
{ Lineas=new[]{new LineaPlanificadaV1("Acero",21)} },default));
        Assert.Equal(20,(await db.Planificaciones.SingleAsync()).CantidadTotal);
    }
    [Fact]
    public async Task Consumidores_concurrentes_guardan_un_solo_efecto()
    {
        await using var bd=BaseSqlServer.Crear();await using var a=Inventario(bd);await
a.Database.EnsureCreatedAsync();await using var b=Inventario(bd);var e=Evento();
        var resultados=await Task.WhenAll(new ReceptorPlanificacion(a).ProcesarAsync(e,default),new
ReceptorPlanificacion(b).ProcesarAsync(e,default));
        Assert.Single(resultados, x=>x);await using var leer=Inventario(bd);Assert.Equal(1,await
leer.Planificaciones.CountAsync());
    }

    private sealed class TransportePrueba(bool falla) : ITransporteEventos
    {
        public int Publicados;
        public Task PublicarAsync(string contenido,CancellationToken ct)
        { if(falla)throw new IOException("Broker no disponible");Publicados++;return Task.CompletedTask; }
    }
    [Fact]
    public async Task Fallo_del_broker_conserva_outbox_pendiente()
    {
        await using var bd=BaseSqlServer.Crear();await using var db=Produccion(bd);await
```

```
db.Database.EnsureCreatedAsync();
        var o=new OrdenProduccion("Axia","Acero",20);db.Ordenes.Add(o);await db.SaveChangesAsync();await new
PlanificadorIntegrado(db).PlanificarAsync(o.Id,default);
        await Assert.ThrowsAsync<IOException>(()=>new DespachadorOutbox(db,new
TransportePrueba(true)).DespacharAsync(default));
        await using var leer=Produccion(bd);Assert.Null((await
leer.MensajesSalida.SingleAsync()).EnviadoEnUtc);Assert.Equal(EstadoOrdenProduccion.Planificada,(await
leer.Ordenes.SingleAsync()).Estado);
    }
    [Fact]
    public async Task Confirmacion_del_broker_marca_envio_y_no_repite_pendiente()
    {
        await using var bd=BaseSqlServer.Crear();await using var db=Produccion(bd);await
db.Database.EnsureCreatedAsync();
        var o=new OrdenProduccion("Axia","Acero",20);db.Ordenes.Add(o);await db.SaveChangesAsync();await new
PlanificadorIntegrado(db).PlanificarAsync(o.Id,default);var transporte=new TransportePrueba(false);
        Assert.Equal(1,await new
DespachadorOutbox(db,transporte).DespacharAsync(default));Assert.Equal(0,await new
DespachadorOutbox(db,transporte).DespacharAsync(default));
        await using var leer=Produccion(bd);Assert.NotNull((await
leer.MensajesSalida.SingleAsync()).EnviadoEnUtc);Assert.Equal(1,transporte.Publicados);
    }
    [Fact]
    public async Task Dos_contextos_con_lectura_antigua_no_planifican_dos_veces()
    {
        await using var bd=BaseSqlServer.Crear();await using var a=Produccion(bd);await
a.Database.EnsureCreatedAsync();
        var o=new OrdenProduccion("Axia","Acero",20);a.Ordenes.Add(o);await a.SaveChangesAsync();await using
var b=Produccion(bd);
        await b.Ordenes.Include(x=>x.Lineas).SingleAsync();Assert.Equal(EstadoPlanificacion.Aceptada,await
new PlanificadorIntegrado(a).PlanificarAsync(o.Id,default));
        Assert.Equal(EstadoPlanificacion.Conflicto,await new
PlanificadorIntegrado(b).PlanificarAsync(o.Id,default));await using var
leer=Produccion(bd);Assert.Equal(1,await leer.MensajesSalida.CountAsync());
    }

    [Theory]
    [InlineData(true)]
    [InlineData(false)]
    public async Task Anadir_linea_y_planificar_comparten_version_del_agregado(bool planificarPrimero)
    {
        await using var bd = BaseSqlServer.Crear();
        await using var a = Produccion(bd); await a.Database.EnsureCreatedAsync();
        var orden = new OrdenProduccion("Axia", "Acero", 20);
        a.Ordenes.Add(orden); await a.SaveChangesAsync();
        await using var b = Produccion(bd);
        await b.Ordenes.Include(x => x.Lineas).SingleAsync();
        var casos = new OrdenesProduccionCasosUso(new OrdenProduccionRepositorio(a), new
ProduccionUnitOfWork(a));
        if (planificarPrimero)
        {
            Assert.Equal(EstadoPlanificacion.Aceptada, await new
PlanificadorIntegrado(b).PlanificarAsync(orden.Id, default));
            var resultado = await casos.AgregarLineaAsync(orden.Id, "Acero", 10);
            Assert.Equal(TipoErrorProduccion.Conflicto, resultado.Error!.Tipo);
        }
        else
        {
            var resultado = await casos.AgregarLineaAsync(orden.Id, "Acero", 10);
            Assert.True(resultado.EsExito);
            Assert.Equal(EstadoPlanificacion.Conflicto, await new
PlanificadorIntegrado(b).PlanificarAsync(orden.Id, default));
            await using var reintento = Produccion(bd);
            Assert.Equal(EstadoPlanificacion.Aceptada, await new
PlanificadorIntegrado(reintento).PlanificarAsync(orden.Id, default));
        }
        await using var leer = Produccion(bd);
        var persistida = await leer.Ordenes.Include(x => x.Lineas).SingleAsync();
        var evento = JsonSerializer.Deserialize<OrdenPlanificadaV1>((await
leer.MensajesSalida.SingleAsync()).Contenido)!;
        Assert.Equal(EstadoOrdenProduccion.Planificada, persistida.Estado);
        Assert.Equal(planificarPrimero ? 20 : 30, persistida.CantidadTotal);
        Assert.Equal(persistida.CantidadTotal, evento.Lineas.Sum(x => x.Cantidad));
    }
}
```

Guarda los datos de preparación o la modificación que esta prueba comprobará desde SQL Server real.

Implementación funcional: `RabbitMqTests.cs`

Estas pruebas exigen un RabbitMQ accesible o arrancan uno con Testcontainers. Publican y consumen mensajes reales, comprueban reentrega, cola de errores y repetición tras publicar sin lograr marcar el outbox. Eliminan solo su topología con nombre único.

Archivo completo: `tests/AceriaApp.ComunicacionAsincrona.Tests/RabbitMqTests.cs`. Crea el archivo y sus carpetas si no existen.

```
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Contratos;
using AceriaApp.Comunicacion.Infraestructura;
using AceriaApp.Inventario.Api;
using AceriaApp.Produccion.Dominio;
using AceriaApp.Produccion.Infraestructura;
using AceriaApp.PruebasRabbitMq;
using AceriaApp.PruebasSqlServer;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Infrastructure;
using Xunit;
namespace AceriaApp.ComunicacionAsincrona.Tests;
public class RabbitMqTests
{
    [Fact]
    public async Task Broker_real_conserva_mensaje_sin_consumidor_y_lo_entrega()
    {
        await using var entorno=await RabbitPruebas.CrearAsync();
        var e=new OrdenPlanificadaV1(Guid.NewGuid(),Guid.NewGuid(),DateTimeOffset.UtcNow,new[]{new
LineaPlanificadaV1("Acero",10)});
        await entorno.Bus.PublicarAsync(JsonSerializer.Serialize(e),default);
        string? contenido=null;
        Assert.True(await entorno.Bus.ConsumirUnoAsync((json,ct)=>{contenido=json;return
Task.CompletedTask;},default));
        Assert.Equal(e.EventoId,JsonSerializer.Deserialize<OrdenPlanificadaV1>(contenido!)!.EventoId);
        Assert.False(await entorno.Bus.ConsumirUnoAsync((_,_)=>Task.CompletedTask,default));
    }
    [Fact]
    public async Task Fallo_tecnico_reencola_para_otro_intento()
    {
        await using var entorno=await RabbitPruebas.CrearAsync();await
entorno.Bus.PublicarAsync("{}",default);
        await Assert.ThrowsAsync<IOException>(()=>entorno.Bus.ConsumirUnoAsync((_,_)=>throw new
IOException("SQL temporalmente no disponible"),default));
        Assert.True(await entorno.Bus.ConsumirUnoAsync((_,_)=>Task.CompletedTask,default));
    }
    [Fact]
    public async Task Mensaje_incompatible_va_a_cola_de_errores()
    {
        await using var entorno=await RabbitPruebas.CrearAsync();await
entorno.Bus.PublicarAsync("incompatible",default);
        Assert.True(await entorno.Bus.ConsumirUnoAsync((_,_)=>throw new InvalidDataException("Version
incompatible"),default));
        Assert.False(await entorno.Bus.ConsumirUnoAsync((_,_)=>Task.CompletedTask,default));
        Assert.Equal(1u,await entorno.Bus.ContarErroresAsync(default));
    }

    [Fact]
    public async Task Corte_tras_publish_confirmado_republica_mismo_evento_y_SQL_aplica_un_solo_efecto()
    {
        await using var entorno = await RabbitPruebas.CrearAsync();
        await using var baseProduccion = BaseSqlServer.Crear();
        await using var baseInventario = BaseSqlServer.Crear();
        await using (var inventario = InventarioDelTest(baseInventario))
            await inventario.Database.EnsureCreatedAsync();

        Guid ordenId;
        Guid eventoId;
```

```
        string contenidoOriginal;
        await using (var preparar = ProduccionDelTest(baseProduccion))
        {
            await preparar.Database.EnsureCreatedAsync();
            var orden = new OrdenProduccion("Axia", "Acero", 20);
            preparar.Ordenes.Add(orden);
            await preparar.SaveChangesAsync();
            ordenId = orden.Id;
            Assert.Equal(EstadoPlanificacion.Aceptada,
                await new PlanificadorIntegrado(preparar).PlanificarAsync(ordenId, default));
            var salida = await preparar.MensajesSalida.AsNoTracking().SingleAsync();
            eventoId = salida.Id;
            contenidoOriginal = salida.Contenido;
            Assert.Null(salida.EnviadoEnUtc);
            await preparar.Database.ExecuteSqlRawAsync("""
                CREATE TRIGGER dbo.RechazarMarcaDeEnvio ON dbo.MensajesSalida AFTER UPDATE AS
                BEGIN
                    SET NOCOUNT ON;
                    IF UPDATE(EnviadoEnUtc)
                        THROW 51002, 'Fallo inducido despues del publish confirmado', 1;
                END;
                """);
        }

        await using (var primerIntento = ProduccionDelTest(baseProduccion))
        {
            var fallo = await Assert.ThrowsAsync<DbUpdateException>(() =>
                new DespachadorOutbox(primerIntento, entorno.Bus).DespacharAsync(default));
            Assert.Equal(51002, Assert.IsType<SqlException>(fallo.InnerException).Number);
        }
        await using (var pendiente = ProduccionDelTest(baseProduccion))
        {
            var salida = await pendiente.MensajesSalida.AsNoTracking().SingleAsync();
            Assert.Equal(eventoId, salida.Id);
            Assert.Equal(contenidoOriginal, salida.Contenido);
            Assert.Null(salida.EnviadoEnUtc);
            Assert.Equal(EstadoOrdenProduccion.Planificada,
                (await pendiente.Ordenes.AsNoTracking().SingleAsync()).Estado);
            await pendiente.Database.ExecuteSqlRawAsync("DROP TRIGGER dbo.RechazarMarcaDeEnvio;");
        }

        await using (var segundoIntento = ProduccionDelTest(baseProduccion))
            Assert.Equal(1, await new DespachadorOutbox(segundoIntento,
entorno.Bus).DespacharAsync(default));
        await using (var enviado = ProduccionDelTest(baseProduccion))
        {
            var salida = await enviado.MensajesSalida.AsNoTracking().SingleAsync();
            Assert.Equal(eventoId, salida.Id);
            Assert.Equal(contenidoOriginal, salida.Contenido);
            Assert.NotNull(salida.EnviadoEnUtc);
            Assert.Equal(0, await new DespachadorOutbox(enviado, entorno.Bus).DespacharAsync(default));
        }

        var entregas = new List<OrdenPlanificadaV1>();
        var efectos = new List<bool>();
        async Task ProcesarEntrega(string contenido, CancellationToken ct)
        {
            Assert.Equal(contenidoOriginal, contenido);
            var evento = JsonSerializer.Deserialize<OrdenPlanificadaV1>(contenido)!;
            await using var inventario = InventarioDelTest(baseInventario);
            efectos.Add(await new ReceptorPlanificacion(inventario).ProcesarAsync(evento, ct));
            entregas.Add(evento);
        }
        await ConsumirEntregaAsync(entorno.Bus, ProcesarEntrega);
        await ConsumirEntregaAsync(entorno.Bus, ProcesarEntrega);
        Assert.Equal(2, entregas.Count);
        Assert.All(entregas, evento =>
        {
            Assert.Equal(eventoId, evento.EventoId);
            Assert.Equal(ordenId, evento.OrdenId);
            Assert.Equal(20, evento.Lineas.Sum(linea => linea.Cantidad));
        });
        Assert.Equal(new[] { true, false }, efectos);
        Assert.False(await entorno.Bus.ConsumirUnoAsync(ProcesarEntrega, default));
        Assert.Equal(0u, await entorno.Bus.ContarErroresAsync(default));
```

```
        await using var comprobar = InventarioDelTest(baseInventario);
        var efecto = Assert.Single(await comprobar.Planificaciones.AsNoTracking().ToListAsync());
        Assert.Equal(eventoId, efecto.EventoId);
        Assert.Equal(ordenId, efecto.OrdenId);
        Assert.Equal(20, efecto.CantidadTotal);
        var huellaEsperada = Convert.ToHexString(SHA256.HashData(Encoding.UTF8.GetBytes(contenidoOriginal)));
        Assert.Equal(huellaEsperada, efecto.HuellaContenido);
        Assert.Equal("Microsoft.EntityFrameworkCore.SqlServer", comprobar.Database.ProviderName);
    }

    private static ProduccionDbContext ProduccionDelTest(BaseSqlServer bd) => new(
        new DbContextOptionsBuilder<ProduccionDbContext>().UseSqlServer(bd.ConnectionString)
            .ReplaceService<IModelCustomizer, ModeloConDisparador>().Options);

    private static InventarioDbContext InventarioDelTest(BaseSqlServer bd) => new(
        new DbContextOptionsBuilder<InventarioDbContext>().UseSqlServer(bd.ConnectionString).Options);

    private sealed class ModeloConDisparador(ModelCustomizerDependencies dependencias)
        : ModelCustomizer(dependencias)
    {
        public override void Customize(ModelBuilder modelo, DbContext contexto)
        {
            base.Customize(modelo, contexto);
            // Permite que el trigger ejecute THROW; evita el error previo de OUTPUT con triggers.
            modelo.Entity<MensajeSalida>().ToTable("MensajesSalida", tabla =>
tabla.UseSqlOutputClause(false));
        }
    }

    private static async Task ConsumirEntregaAsync(RabbitMqTransporte bus,
        Func<string, CancellationToken, Task> procesar)
    {
        using var plazo = new CancellationTokenSource(TimeSpan.FromSeconds(15));
        using var sondeo = new PeriodicTimer(TimeSpan.FromMilliseconds(50));
        while (!await bus.ConsumirUnoAsync(procesar, plazo.Token))
            await sondeo.WaitForNextTickAsync(plazo.Token);
    }
}
```

Implementación funcional: `RabbitPruebas.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `tests/Compartido/RabbitPruebas.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Infraestructura;
using Testcontainers.RabbitMq;
namespace AceriaApp.PruebasRabbitMq;
public sealed class RabbitPruebas : IAsyncDisposable
{
    private static readonly Lazy<Task<RabbitMqContainer>> Local = new(async () =>
    {
        var contenedor = new RabbitMqBuilder().WithImage("rabbitmq:4.1.5-management")
            .WithUsername("curso").WithPassword("curso-local").Build();
        await contenedor.StartAsync();return contenedor;
    });
    public RabbitMqTransporte Bus { get; }
    private RabbitPruebas(string uri) { Bus = new(new RabbitMqOpciones(uri, "prueba-" +
Guid.NewGuid().ToString("N"))); }
    public static async Task<RabbitPruebas> CrearAsync()
    {
        var uri = Environment.GetEnvironmentVariable("ACERIA_TEST_RABBITMQ");
        if (string.IsNullOrWhiteSpace(uri)) uri = (await Local.Value).GetConnectionString();
        return new RabbitPruebas(uri);
    }
    public async ValueTask DisposeAsync()
    {
        try { await Bus.EliminarTopologiaAsync(default); }
        finally { await Bus.DisposeAsync(); }
```

```
    }
}
```

Libera recursos del adaptador o de la prueba al terminar su ámbito.

PASO 6.2.16 — Compilar la solución completa

En esta etapa el resultado debe compilar y superar las pruebas de mensajería con un broker accesible. Sin procesar eventos y comprobar sus efectos no consideraremos demostrado el recorrido asíncrono.

1. Aplica los bloques de integración funcional de este paso en el orden mostrado.
2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 28 proyectos y termina sin errores.

Los archivos funcionales de este punto se han incorporado en los pasos 5, 8, 9, 10, 13 y 15. Antes de compilar, agrega todos los proyectos nuevos a la solución con los comandos indicados y guarda los archivos completos.

Verificación concreta. La solución de la etapa compila con 28 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

PASO 6.2.17 — Ejecutar todos los tests del punto 6.2

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release --no-build --no-restore`.
2. Comprueba que se ejecutan los 13 tests de catálogo y las pruebas funcionales añadidas al proyecto hasta el paso 15.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 13 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 6.2.18 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.

3. Suma los resultados de los archivos TRX y verifica 264 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04 y M05: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 264 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

Comprobación con procesos reales

Utiliza las dos terminales y las direcciones del apartado «Arranque de los dos procesos». Comprueba que Inventario escucha en 5106 y Producción en 5105. Los tests de cliente con un handler controlado y los de WebApplicationFactory no reemplazan este recorrido por sockets.

Crea una orden desde otra terminal; no uses el endpoint de planificación anterior de M5 para este recorrido:

```
$orden=Invoke-RestMethod -Method Post -Uri http://localhost:5105/api/v1/produccion/ordenes -ContentType
'application/json' -Body '{"cliente":"Axia","material":"Acero","cantidad":20}'
$id=$orden.id
Invoke-RestMethod -Method Post -Uri "http://localhost:5105/api/v1/comunicacion/ordenes/$id/planificar"
Invoke-RestMethod -Uri "http://localhost:5105/api/v1/produccion/ordenes/$id"
Invoke-RestMethod -Uri "http://localhost:5106/api/v1/planificaciones/$id"
```

La planificación devuelve 202. La orden queda `Planificada`; la última consulta puede devolver 404 mientras el evento siga pendiente y 200 tras procesarlo. Repite esa consulta durante unos segundos, con un límite de espera explícito; una demora no es garantía de entrega futura. Con los workers activos, la proyección muestra `cantidadTotal=20`. Detén y vuelve a arrancar Inventario: esa proyección debe seguir en SQL. Publicador y consumidor reintentan los fallos técnicos con pausa; revisa sus logs si el estado no avanza. Los mensajes incompatibles van a la cola terminada en `.errores`; un operador debe examinarlos antes de un reproceso deliberado.

La llamada a `/api/v1/produccion/ordenes/{id}/planificar` sigue siendo el ejercicio local heredado de M5 y no genera eventos. El recorrido integrado usa siempre `/api/v1/comunicacion/ordenes/{id}/planificar`. Mantener ambas rutas permite comparar sus contratos sin atribuir mensajería a una operación que no la implementa.

PASO 6.2.19 — Actualizar el README del punto 6.2

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `6.2-comunicacion-asincrona-eventos`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `6.1-comunicacion-sincrona-http` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 6.2 — Comunicación asíncrona basada en eventos

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base
anterior.
```

```
## Estado del proyecto

- Solución `AceriaApp.sln`: 28 proyectos y 264 pruebas acumuladas.
- 13 pruebas nuevas de análisis en `tests/AceriaApp.ComunicacionAsincrona.Tests`.
- Tipos `Evento`, `ValidadorComunicacionAsincrona` y `CatalogoEventos` en `src/AceriaApp.Api/
ComunicacionAsincrona`.
- Los catálogos no demuestran tráfico HTTP ni entrega de mensajes.

## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M06 para iniciar los adaptadores y observar cada recorrido de comunicación.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/comunicacion-asincrona-eventos.md` y el ADR-025. Los
bloques funcionales de los pasos 5 a 15 completan la integración del punto. Se conservan los documentos y
pruebas heredados.

## Documentación

- [Tema del punto](docs/comunicacion-asincrona-eventos.md).
- [ADR-025](docs/ADR-025-comunicacion-asincrona-eventos.md).
- Los 30 documentos anteriores, incluidos los ADR-010 a ADR-024, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 28 proyectos, 264 pruebas y 13 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 6.2.20 — Crear el ADR-025 del punto 6.2

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-025-comunicacion-asincrona-eventos.md` dentro de la copia de `6.2-comunicacion-asincrona-eventos`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-025-comunicacion-asincrona-eventos.md` existe y que los 30 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-025-comunicacion-asincrona-eventos.md`. **Operación:** Crear.

```
# ADR-025: Comunicación asíncrona basada en eventos

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita razonar explícitamente sobre sus interacciones. La solución
conserva los modelos y pruebas heredados; el catálogo nuevo es un ejercicio de análisis separado de los
adaptadores de comunicación ejecutables.

## Decisión

Separar eventos locales de dominio de contratos de integración. Para el catálogo de integración del ejercicio
declarar consumidores, idempotencia y política de reintento. La implementación de RabbitMQ verifica
publicación y consumo; un booleano no establece una garantía de entrega.

## Alternativas consideradas
```

```
- Elegir un solo estilo para todo el sistema: descartado porque las interacciones tienen necesidades
diferentes.
- Omitir los criterios de recuperación y consistencia: descartado porque deja ambiguos los fallos parciales.
- Considerar suficiente la validación del catálogo: descartado porque no comprueba la comunicación
ejecutable.

## Consecuencias

- Positivas: decisiones explícitas, casos de rechazo reproducibles y trazabilidad por interacción.
- Negativas: mantener contratos, políticas y pruebas de comportamiento añade trabajo.
- Mitigación: distinguir pruebas de análisis y pruebas funcionales, mantener límites claros y revisar las
decisiones con evidencia operativa.
```

Verificación concreta. `docs/ADR-025-comunicacion-asincrona-eventos.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-025-comunicacion-asincrona-eventos.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 6.2.21 — Verificar que no se ha modificado ningún archivo del punto 6.1

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.1-comunicacion-sincrona-http` y actual `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.
3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 30 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 30 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 6.2.22 — Verificar la estructura del punto 6.2

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, sin proyectos ocultos fuera de la etapa.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/ComunicacionAsincrona` y las dos clases de tests en `tests/AceriaApp.ComunicacionAsincrona.Tests`.
3. Comprueba los 32 documentos de `docs`, incluida la ficha del punto y el ADR-025.
4. Ejecuta `dotnet sln AceriaApp.sln list` y contrasta sus 28 proyectos con los `.csproj` presentes.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 28 proyectos.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 6.2.23 — Verificar el `.gitignore` del punto 6.2

Las exclusiones se aplican desde la raíz a las etapas; no es necesario inventar un `.gitignore` propio dentro de cada copia.

1. Abre el `.gitignore` de la raíz del repositorio del curso.
2. Comprueba las reglas `**/bin/`, `**/obj/` y `**/TestResults/` que ya excluyen los artefactos de todas las etapas.
3. Ejecuta `git status --short` y revisa que no aparezcan binarios, cachés ni resultados temporales.
4. Conserva las exclusiones existentes; si aparecen archivos locales de Visual Studio, excluye `.vs/`, `*.user` y `*.suo` mediante `.git/info/exclude`, sin ignorar código ni configuración entregable.

Verificación concreta. Los archivos compilados y resultados de tests están excluidos; los archivos del punto siguen visibles para Git.

Diagnóstico si falla.

- Ignorar todo `tests/` elimina la evidencia reproducible: excluye solo sus artefactos generados.
- Esperar un `.gitignore` dentro de cada etapa puede llevar a crear duplicados innecesarios: revisa primero el de la raíz.

PASO 6.2.24 — Hacer el commit del punto 6.2 y verificar el estado final

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 6.2: comunicación asíncrona basada en eventos`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 6.2 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorComunicacionAsincrona` y una prueba adicional. Trabaja en una copia optativa de `6.2-comunicacion-asincrona-eventos` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 6.2.1 — Definir el contrato del resumen

Abre `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs` e inserta el contrato después del `record` de resultado, en el mismo namespace.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs`. **Operación:** Insertar después del ancla indicada.

Ancla exacta: `public record ResultadoComunicacionAsincrona(bool EsValida, IReadOnlyList<string> Errores);`. Debe aparecer una sola vez.

```
public record ResumenEventos(int Total, int DeIntegracion, int DeDominio, int TotalConsumidores);
```

Reto 6.2.3 — Calcular los recuentos

Inserta el método dentro de `ValidadorComunicacionAsincrona`, antes del campo `private static readonly string[]`; conserva las cuatro posiciones del constructor del resumen.

Archivo: `src/AceriaApp.Api/ComunicacionAsincrona/ValidadorComunicacionAsincrona.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `private static readonly string[]`. Debe aparecer una sola vez.

```
    public static ResumenEventos Resumen(IEnumerable<Evento> eventos)
    {
        var lista = eventos.ToList();
        var integracion = lista.Count(e => e.Tipo == "Integracion");
        var dominio = lista.Count(e => e.Tipo == "Dominio");
        var consumidores = lista.Sum(e => e.NumeroDeConsumidores);
        return new ResumenEventos(lista.Count, integracion, dominio, consumidores);
    }
```

Cuenta las entradas que cumplen el criterio indicado. Suma las suscripciones declaradas; un consumidor puede aparecer en varios eventos. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 6.2.6 — Verificar el catálogo por defecto

Inserta la prueba dentro de `ValidadorComunicacionAsincronaTests`, antes del campo `_validador`. La prueba comprueba total 4, integración 4, dominio 0 y suma de suscripciones 6.

Archivo: `tests/AceriaApp.ComunicacionAsincrona.Tests/ValidadorComunicacionAsincronaTests.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `private readonly ValidadorComunicacionAsincrona _validador = new();`. Debe aparecer una sola vez.

```
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorComunicacionAsincrona.Resumen(CatalogoEventos.EventosPorDefecto());

        resumen.Total.Should().Be(4);
        resumen.DeIntegracion.Should().Be(4);
        resumen.DeDominio.Should().Be(0);
        resumen.TotalConsumidores.Should().Be(6);
    }
```

Calcula el resumen sobre el catálogo por defecto. Compara el recuento obtenido con el valor esperado del catálogo.

Comprobación del reto. Compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.ComunicacionAsincrona.Tests/AceriaApp.ComunicacionAsincrona.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 13 a 14 tests; la solución debe ejecutar 264 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que la etapa obligatoria conserva sus hashes.

Diagnóstico si falla. Si el método queda fuera de la clase, mueve solo el fragmento al ancla señalada. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 13 pruebas de catálogo, los adaptadores funcionales y 32 documentos Markdown. La solución incluye 28 proyectos y 264 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.

Punto 6.3 — Cuándo utilizar cada enfoque

Seleccionar HTTP o eventos a partir de una necesidad concreta

La comunicación síncrona y la asíncrona resuelven problemas diferentes. Una consulta que necesita respuesta inmediata puede requerir HTTP; un cambio de estado que puede procesarse más tarde encaja en un evento cuando existe una estrategia de entrega e idempotencia. Para decidir tendremos en cuenta latencia, disponibilidad, consistencia y recuperación ante errores.

El catálogo de decisiones del punto recoge escenarios de AceriaApp y permite justificar cada elección, sin convertir sus indicadores en medidas automáticas del sistema desplegado. La conclusión se contrasta con los recorridos reales construidos en 6.1 y 6.2, de modo que los criterios de diseño no sustituyen la evidencia de HTTP, RabbitMQ y SQL Server.

Carpeta de trabajo: `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque`.

Objetivo: completar los veinticuatro pasos, conservar la etapa anterior y comprobar por separado el análisis y la comunicación ejecutable. La etapa termina con 283 pruebas en 29 proyectos.

Ejercicio guiado. Incorpora `DecisionComunicacion`, `ValidadorDecisiones` y `CatalogoDecisiones`, dos documentos nuevos y 12 pruebas de catálogo. Los bloques de integración de los pasos 5 a 15 implementan el comportamiento remoto.

PASO 6.3.1 — Crear la carpeta del punto 6.3 a partir del punto 6.2

La etapa autónoma continúa 28 proyectos y 264 pruebas antes de añadir los cambios del punto.

1. Localiza la etapa completa `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos`; comprueba que contiene `AceriaApp.sln`.
2. Crea `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque` y copia en ella el contenido de `6.2-comunicacion-asincrona-eventos`; en Windows usa `xcopy` con `/E /I /H /Y`.
3. Excluye `bin`, `obj`, `TestResults` y `.vs`; trabaja solamente dentro de `6.3-cuando-usar-cada-enfoque`.
4. Comprueba con `dir` que aparecen `AceriaApp.sln`, `README.md`, `src`, `tests`, `docs` y `global.json`.

Verificación concreta. La copia corresponde a `6.2-comunicacion-asincrona-eventos` y conserva todas sus carpetas de código y documentación.

Diagnóstico si falla.

- Usar `copy` no copia carpetas recursivamente: usa `xcopy` con los parámetros indicados.
- Copiar otra etapa pierde las pruebas de `6.2-comunicacion-asincrona-eventos`: verifica la ruta antes de editar.

PASO 6.3.2 — Abrir la solución copiada en Visual Studio con soporte .NET 8

Compilar la copia detecta rutas y dependencias incompletas antes de añadir archivos.

1. Abre el `AceriaApp.sln` de `6.3-cuando-usar-cada-enfoque` en Visual Studio.
2. Ejecuta `dotnet sln AceriaApp.sln list` y comprueba los 28 proyectos heredados.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release` desde la carpeta de esta etapa.
4. Comprueba que la compilación termina sin errores; si hay DLL bloqueadas, detén los procesos que las utilizan.

Verificación concreta. La solución lista 28 proyectos y la compilación termina correctamente.

Diagnóstico si falla.

- Abrir la solución de la etapa anterior dirige los cambios a una carpeta equivocada: revisa su ruta en Visual Studio.
- Una restauración fallida necesita revisar el SDK y la conectividad a NuGet; no se corrige cambiando versiones al azar.

PASO 6.3.3 — Verificar que la carpeta `docs` contiene los documentos del punto 6.2

El punto conserva 32 documentos reales y añadirá dos; el primer ADR disponible es ADR-010.

1. Expande `docs` y comprueba sus 32 archivos Markdown.
2. Localiza los ADR-010 a ADR-025 y las fichas temáticas heredadas.
3. Compara nombres y contenido con `docs` de `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos`.
4. Comprueba que aún no existen `docs/cuando-usar-cada-enfoque.md` ni `docs/ADR-026-cuando-usar-cada-enfoque.md`.

Verificación concreta. Hay 32 documentos heredados idénticos; todavía no existen los dos documentos del punto.

Diagnóstico si falla.

- Dar por heredados ADR-001 a ADR-009 crea enlaces rotos: enumera los documentos reales.
- Eliminar una ficha rompe la conservación: restaura únicamente el archivo afectado desde la etapa anterior.

PASO 6.3.4 — Crear el documento de criterios de decisión en `docs`

Antes de elegir el transporte conviene escribir qué tolera el negocio: latencia, indisponibilidad, consistencia eventual y necesidad de respuesta. La decisión se justifica con esos requisitos, no por preferir una tecnología.

1. Crea `docs/cuando-usar-cada-enfoque.md` dentro de la copia de `6.3-cuando-usar-cada-enfoque`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.

4. Comprueba que `docs/cuando-usar-cada-enfoque.md` existe y que los 32 documentos anteriores permanecen intactos.

Archivo: `docs/cuando-usar-cada-enfoque.md`. **Operación:** Crear.

```
# Cuándo utilizar cada enfoque — Punto 6.3

## Decisión por interacción

Selecciona comunicación síncrona, asíncrona o una combinación según el resultado que necesita el caso de uso. Un mismo servicio puede utilizar los tres estilos en operaciones diferentes.

## Respuesta inmediata

HTTP facilita recibir una respuesta directa. La asincronía de `await` no cambia ese contrato. Una notificación asíncrona sin protocolo de respuesta no puede ofrecer el resultado inmediato del consumidor. Un acuse de recibo del broker no es el resultado de negocio.

## Consistencia

La necesidad de conocer el stock actual no hace atómica una reserva posterior. La consistencia eventual requiere estados pendientes y una forma de observar el avance. HTTP por sí solo tampoco coordina una transacción distribuida.

## Disponibilidad y fallos

HTTP depende de la disponibilidad simultánea del destinatario. Mensajería puede desacoplar el consumo si existe almacenamiento durable, pero añade dependencias operativas y retrasos. Documenta timeout, reintentos seguros, mensajes fallidos y recuperación.

## Volumen, frecuencia y latencia

Mide las tasas de llegada, capacidad del consumidor, tamaño del mensaje y latencia admitida. Una cola absorbe picos limitados; no resuelve una falta de capacidad permanente.

## Organización

Considera la autonomía de los equipos, la propiedad de los contratos, las versiones compatibles y la capacidad de operación del broker. Ningún estilo exige automáticamente un microservicio nuevo.

## Patrones híbridos

Combina consulta síncrona con notificación asíncrona cuando cada fase lo necesita. Describe el protocolo, sus estados y la autoridad de cada dato; un indicador booleano no implementa una saga.

## Matriz de decisión

| Necesidad | Opción a evaluar | Condición que hay que verificar |
|---|---|---|
| Resultado del destino durante la petición | HTTP | Timeout y tratamiento de indisponibilidad |
| Notificación de un hecho a varios interesados | Eventos | Contratos versionados y entrega durable |
| Escritura local y aviso remoto | Eventos con outbox | Transacción local e idempotencia |
| Consulta actual y trabajo posterior | Híbrida | Estado pendiente y consulta de avance |

## Decisiones del catálogo

Se describen dos interacciones síncronas, dos asíncronas y una híbrida. Una interacción síncrona sin necesidad de respuesta inmediata puede ser válida por simplicidad u operación; no se rechaza por ese único dato. El análisis asíncrono de este ejercicio requiere tolerancia eventual y no promete resultado inmediato del consumidor. La modalidad híbrida exige describir su patrón.

## Cierre del módulo

Los catálogos de llamadas, eventos y decisiones verifican datos de análisis. La integración funcional de HTTP y mensajería aporta pruebas separadas de comportamiento. Ni los nombres de servicios ni los booleanos del catálogo demuestran tráfico, colas, reintentos o transacciones distribuidas.
```

Verificación concreta. `docs/cuando-usar-cada-enfoque.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/cuando-usar-cada-enfoque.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 6.3.5 — Crear la carpeta `Decisiones` en el proyecto API

`Decisiones` agrupa datos y reglas del ejercicio de análisis; los adaptadores de comunicación tienen sus propias responsabilidades.

1. Expande el proyecto `AceriaApp.Api` y localiza `src/AceriaApp.Api`.
2. Crea la carpeta `Decisiones` directamente dentro del proyecto API.
3. Comprueba que `src/AceriaApp.Api/Decisiones` no está anidada dentro de otra carpeta temática.
4. Conserva las carpetas y archivos heredados de `6.2-comunicacion-asincrona-eventos`.

Verificación concreta. Existe `src/AceriaApp.Api/Decisiones` sin alterar las carpetas previas.

Diagnóstico si falla.

- Crear la carpeta dentro de `tests` impide compilar los tipos con la API: comprueba la ruta física.
- Renombrar una carpeta heredada rompe namespaces y enlaces: crea solo la carpeta nueva.

PASO 6.3.6 — Crear la clase `DecisionComunicacion`

DecisionComunicacion reúne los criterios sobre una colaboración. Es un modelo para analizar alternativas y no inicia por sí solo un consumidor ni establece una conexión remota.

1. Crea `src/AceriaApp.Api/Decisiones/DecisionComunicacion.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Decisiones/DecisionComunicacion.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Decisiones;

public class DecisionComunicacion
{
    public string Interaccion { get; init; } = string.Empty;
    public string Enfoque { get; init; } = string.Empty;
    public bool RequiereRespuestaInmediata { get; init; }
    public bool ToleraConsistenciaEventual { get; init; }
    public bool TienePatronHibrido { get; init; }
}
```

Sitúa el tipo en el espacio de nombres que importan las pruebas del punto. Declara el tipo público del ejercicio de análisis. Declara un dato del análisis que se fija al construir la entrada.

Verificación concreta. `DecisionComunicacion` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Decisiones` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.3.7 — Crear la clase `ValidadorDecisiones`

La clasificación del validador depende de los umbrales definidos para el ejercicio. Es necesario explicar por qué recomienda cada enfoque y reconocer los casos que quedan fuera del modelo.

1. Crea `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Decisiones;

public class ValidadorDecisiones
{
    private static readonly string[] EnfoquesPermitidos = { "Sincrona", "Asincrona", "Hibrida" };

    public ResultadoDecisiones Validar(IEnumerable<DecisionComunicacion> decisiones)
    {
        var errores = new List<string>();
        var lista = decisiones.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No hay decisiones declaradas.");
        }

        foreach (var decision in lista)
        {
            if (string.IsNullOrWhiteSpace(decision.Interaccion))
            {
                errores.Add("Una decisión no declara la interacción.");
            }

            if (!EnfoquesPermitidos.Contains(decision.Enfoque))
            {
                errores.Add($"Enfoque no permitido en '{decision.Interaccion}': '{decision.Enfoque}'.");
            }

            if (decision.Enfoque == "Asincrona" && decision.RequiereRespuestaInmediata)
            {
                errores.Add($"La interacción '{decision.Interaccion}' es asíncrona pero requiere respuesta
inmediata del consumidor.");
            }

            if (decision.Enfoque == "Asincrona" && !decision.ToleraConsistenciaEventual)
            {
                errores.Add($"La interacción '{decision.Interaccion}' es asíncrona pero no tolera
consistencia eventual.");
            }

            if (decision.Enfoque == "Hibrida" && !decision.TienePatronHibrido)
            {
                errores.Add($"La interacción '{decision.Interaccion}' es híbrida pero no declara patrón
híbrido.");
            }
        }

        return new ResultadoDecisiones(errores.Count == 0, errores);
    }
}

public record ResultadoDecisiones(bool EsValida, IReadOnlyList<string> Errores);
```

Limita los valores de texto reconocidos por la política didáctica. Expone la operación con el contrato que usan el catálogo o sus pruebas. Inicializa la colección de diagnósticos que se devolverá al finalizar.

Materializa una sola vez la secuencia recibida para recorrerla y contarla. Evalúa una regla concreta antes de añadir el diagnóstico asociado. Añade el diagnóstico de esta regla sin ocultar las demás incidencias.

Revisa cada entrada; un catálogo puede acumular varios diagnósticos. Devuelve validez y diagnósticos; no ejecuta ninguna comunicación remota. Define el resultado inmutable que devuelve el validador o el resumen.

Verificación concreta. `ValidadorDecisiones` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Decisiones` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.3.8 — Añadir los `using` necesarios en `ValidadorDecisiones`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs`.
2. Inserta el bloque de importaciones inmediatamente antes del namespace; comprueba que el ancla aparece una sola vez.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `namespace AceriaApp.Api.Decisiones;`. Debe aparecer una sola vez.

```
using System.Collections.Generic;
using System.Linq;
```

Importa los tipos o extensiones utilizados por el archivo.

Verificación concreta. `ValidadorDecisiones.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

Implementación funcional: `FlujoPlanificacion.cs`

El caso de uso consulta disponibilidad informativa y coordina la planificación local. Agrupa cantidades del mismo material. Entre lectura y planificación puede cambiar el stock: no hay reserva distribuida ni descuento de existencias.

Archivo completo: `src/AceriaApp.Comunicacion.Aplicacion/FlujoPlanificacion.cs`. Crea el archivo y sus carpetas si no existen.

```
namespace AceriaApp.Comunicacion.Aplicacion;
public sealed class FlujoPlanificacion(IPlanificadorIntegrado planificador, IClienteInventario inventario)
{
    public async Task<EstadoPlanificacion> EjecutarAsync(Guid id, CancellationToken ct)
    {
        ct.ThrowIfCancellationRequested();
        var lineas = await planificador.ObtenerLineasAsync(id, ct);
        if (lineas is null) return EstadoPlanificacion.NoExiste;
```

```
        foreach (var grupo in lineas.GroupBy(x => x.Material, StringComparer.OrdinalIgnoreCase))
        {
            var material = await inventario.ObtenerAsync(grupo.Key, ct);
            if (material is null || material.Stock < grupo.Sum(x => x.Cantidad)) return
EstadoPlanificacion.SinDisponibilidad;
        }
        // Esta lectura es informativa: otro proceso puede cambiar el stock. No es una reserva.
        return await planificador.PlanificarAsync(id, ct);
    }
}
```

Detiene la operación si el llamador ya la canceló, antes de producir efectos. Agrupa las líneas del mismo material antes de comparar el total solicitado con el stock informado.

PASO 6.3.9 — Crear la clase `CatalogoDecisiones`

CatalogoDecisiones agrupa escenarios para compararlos de forma consistente. Sus conclusiones no reemplazan las métricas ni los resultados observados en los ensayos de los servicios.

1. Crea `src/AceriaApp.Api/Decisiones/CatalogoDecisiones.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `src/AceriaApp.Api/Decisiones/CatalogoDecisiones.cs`. **Operación:** Crear.

```
namespace AceriaApp.Api.Decisiones;

public class CatalogoDecisiones
{
    public static List<DecisionComunicacion> DecisionesPorDefecto()
    {
        return new List<DecisionComunicacion>
        {
            new() { Interaccion = "Ordenes->Inventario consulta stock", Enfoque = "Sincrona",
RequiereRespuestaInmediata = true, ToleraConsistenciaEventual = false, TienePatronHibrido = false },
            new() { Interaccion = "Ordenes->Clientes consulta cliente", Enfoque = "Sincrona",
RequiereRespuestaInmediata = true, ToleraConsistenciaEventual = false, TienePatronHibrido = false },
            new() { Interaccion = "Ordenes->Inventario reserva material", Enfoque = "Asincrona",
RequiereRespuestaInmediata = false, ToleraConsistenciaEventual = true, TienePatronHibrido = false },
            new() { Interaccion = "Ordenes->Facturacion emision factura", Enfoque = "Asincrona",
RequiereRespuestaInmediata = false, ToleraConsistenciaEventual = true, TienePatronHibrido = false },
            new() { Interaccion = "Ordenes->Inventario notificacion y consulta", Enfoque = "Hibrida",
RequiereRespuestaInmediata = true, ToleraConsistenciaEventual = true, TienePatronHibrido = true }
        };
    }
}
```

Devuelve una lista nueva en cada llamada; modificarla no altera otros casos. Construye una entrada del catálogo o del caso de prueba con valores explícitos. Cierra la colección y termina su inicialización.

Verificación concreta. `CatalogoDecisiones` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un namespace diferente de `AceriaApp.Api.Decisiones` en las clases impide resolverlas desde las pruebas: usa el indicado.
- Confundir un booleano con una política ejecutada lleva a conclusiones incorrectas: limita la verificación a los datos que recorre el validador.

PASO 6.3.10 — Añadir el `using` necesario en `CatalogoDecisiones`

Las importaciones explícitas muestran dónde se declaran los tipos genéricos y las extensiones LINQ, aunque el proyecto tenga importaciones implícitas.

1. Abre `src/AceriaApp.Api/Decisiones/CatalogoDecisiones.cs`.
2. Inserta el bloque de importaciones inmediatamente antes del namespace; comprueba que el ancla aparece una sola vez.
3. Guarda el archivo y comprueba que no has duplicado ningún `using`.
4. Ejecuta `dotnet build src/AceriaApp.Api/AceriaApp.Api.csproj -c Release` y revisa la salida.

Archivo: `src/AceriaApp.Api/Decisiones/CatalogoDecisiones.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `namespace AceriaApp.Api.Decisiones;`. Debe aparecer una sola vez.

```
using System.Collections.Generic;
```

Verificación concreta. `CatalogoDecisiones.cs` conserva el contenido previo y añade solo las importaciones indicadas.

Diagnóstico si falla.

- Pegar las importaciones dentro de una clase no es válido: insértalas antes del namespace.
- Insertar dos veces el bloque produce importaciones duplicadas: comprueba el contenido antes de guardarlo.

PASO 6.3.11 — Crear el proyecto de tests del punto 6.3

Definir el proyecto completo fija .NET 8 y las versiones del runner sin depender de la versión local de la plantilla.

1. Crea la carpeta `tests/AceriaApp.Decisiones.Tests` y el `.csproj` completo del bloque siguiente.
2. No generes `UnitTest1.cs`; los pasos 14 y 15 incluyen las clases de prueba completas.
3. Ejecuta `dotnet sln AceriaApp.sln add tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj` y `dotnet restore tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj`.
4. Ejecuta `dotnet build tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release` y comprueba que el nuevo proyecto aparece en la solución.

Archivo: `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj`. **Operación:** Crear.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
  </ItemGroup>
</Project>
```

Verificación concreta. Existe `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj` y la solución lo incluye una sola vez.

Diagnóstico si falla.

- Crear un proyecto MSTest no descubre los atributos de xUnit: conserva los paquetes mostrados.
- Conservar `UnitTest1.cs` añade una prueba ajena al recuento: no crees ese archivo.

PASO 6.3.12 — Añadir la referencia del proyecto de tests a la API

Las pruebas consumen los tipos de análisis de la API; la API no debe depender de su proyecto de pruebas.

1. Abre `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj`.
2. Reemplaza su contenido completo por el bloque siguiente, que incorpora la referencia a la API.
3. Ejecuta `dotnet restore tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj` y `dotnet build tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release`.
4. Comprueba que `ProjectReference` apunta a `../../src/AceriaApp.Api/AceriaApp.Api.csproj`.

Archivo: `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El proyecto resuelve la referencia a la API y compila sin dependencias circulares.

Diagnóstico si falla.

- Invertir la referencia introduce una dependencia de producción hacia pruebas: añade la referencia en el `.csproj` de tests.
- Una ruta relativa equivocada produce un proyecto no encontrado: resuelve la ruta desde la carpeta de este `.csproj`.

PASO 6.3.13 — Añadir el paquete `FluentAssertions` al proyecto de tests

La versión fijada mantiene reproducibles las aserciones `Should()` de los catálogos.

1. Abre `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj` y conserva las versiones de los paquetes existentes.
2. Reemplaza su contenido por el bloque completo que añade `FluentAssertions` 6.12.2.
3. Ejecuta `dotnet restore tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj` y `dotnet list tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj package`.
4. Comprueba la versión 6.12.2 y ejecuta `dotnet build tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release`.

Archivo: `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj`. **Operación:** Reemplazar el contenido completo.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <IsPackable>false</IsPackable>
    <IsTestProject>true</IsTestProject>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
  </ItemGroup>
</Project>
```

Verificación concreta. El listado de paquetes muestra FluentAssertions 6.12.2 y el proyecto compila.

Diagnóstico si falla.

- Instalar una versión sin fijarla puede cambiar la API y las condiciones de uso: conserva 6.12.2.
- Añadir el paquete a la API no establece la referencia del test: modifica el proyecto de pruebas indicado.

Implementación funcional: `ComunicacionEndpoints.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/Comunicacion/ComunicacionEndpoints.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Comunicacion.Aplicacion;
namespace AceriaApp.Api.Comunicacion;
public static class ComunicacionEndpoints
{
    public static void MapComunicacion(this WebApplication app)
    {
        app.MapGet("/api/v1/comunicacion/materiales/{material}", async (string material, IClienteInventario
inventario, CancellationToken ct) =>
        {
            try
            {
                var dto = await inventario.ObtenerAsync(material, ct);
                return dto is null ? Results.NotFound() : Results.Ok(dto);
            }
            catch (InventarioNoDisponibleException)
            { return Results.Problem(statusCode: 503, title: "Inventario no está disponible."); }
        }).WithTags("Comunicacion");
        app.MapPost("/api/v1/comunicacion/ordenes/{id:guid}/planificar", async (Guid id, FlujoPlanificacion
flujo, CancellationToken ct) =>
        {
            EstadoPlanificacion estado;
            try { estado = await flujo.EjecutarAsync(id, ct); }
            catch (InventarioNoDisponibleException)
            { return Results.Problem(statusCode: 503, title: "Inventario no está disponible."); }
            return estado switch
            {
                EstadoPlanificacion.NoExiste => Results.NotFound(),
                EstadoPlanificacion.SinDisponibilidad => Results.Problem(statusCode: 409, title:
"Disponibilidad informativa insuficiente."),
                EstadoPlanificacion.Conflicto => Results.Problem(statusCode: 409, title: "La orden no está en
borrador."),
                _ => Results.Accepted($"/api/v1/produccion/ordenes/{id}", new { ordenId = id, estado =
"Planificada", propagacion = "Pendiente" })
            };
        }).WithTags("Comunicacion");
```

```
    }
}
```

Publica una consulta HTTP con un contrato de respuesta verificable. Publica la operación de planificación; el adaptador traduce el resultado del caso de uso a HTTP. Devuelve 202: la planificación local está confirmada y la propagación a Inventario es eventual.

Implementación funcional: `appsettings.json`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Api/appsettings.json`. Sustituye el contenido de la copia actual.

```
{
  "ConnectionStrings": {
    "AceriaApp": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaAppM063;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaHexagonal": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaHexagonalM063;Trusted_Connection=True;TrustServerCertificate=True",
    "AceriaProduccion": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaProduccionM063;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m063",
    "WorkersActivos": false
  }
}
```

Separa los almacenes de esta etapa y conserva los de los módulos anteriores.

Implementación funcional: `RegistroComunicacion.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Comunicacion.Infraestructura/RegistroComunicacion.cs`. Sustituye el contenido de la copia actual.

```
using AceriaApp.Comunicacion.Aplicacion;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
namespace AceriaApp.Comunicacion.Infraestructura;
public static class RegistroComunicacion
{
    public static IServiceCollection AddComunicacion(this IServiceCollection services, IConfiguration config)
    {
        services.AddHttpClient<IClienteInventario, ClienteInventarioHttp>(http =>
        {
            http.BaseAddress = new Uri(config["Comunicacion:InventarioUrl"] ?? "http://localhost:5106/");
            http.Timeout = TimeSpan.FromSeconds(3);
        });
        services.AddScoped<FlujoPlanificacion>();
        services.AddScoped<IPlanificadorIntegrado, PlanificadorIntegrado>();
        services.AddScoped<DespachadorOutbox>();
        services.AddSingleton(new RabbitMqOpciones(config["Comunicacion:RabbitMqUri"] ?? "amqp://curso:curso-
local@localhost:5672/",
            config["Comunicacion:Prefijo"] ?? "aceria"));
        services.AddSingleton<RabbitMqTransporte>();
        services.AddSingleton<ITransporteEventos>(sp => sp.GetRequiredService<RabbitMqTransporte>());
        if (config.GetValue<bool>("Comunicacion:WorkersActivos"))
services.AddHostedService<PublicadorOutboxWorker>();
        return services;
```

```
    }
}
```

Registra el cliente tipado y centraliza dirección y timeout; los handlers se administran mediante la factoría. Habilita el proceso periódico únicamente cuando la configuración lo solicita.

Implementación funcional: `appsettings.json`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `src/AceriaApp.Inventario.Api/appsettings.json`. Sustituye el contenido de la copia actual.

```
{
  "ConnectionStrings": {
    "AceriaInventario": "Server=(localdb)\
\MSSQLLocalDB;Database=AceriaInventarioM063;Trusted_Connection=True;TrustServerCertificate=True"
  },
  "Comunicacion": {
    "InventarioUrl": "http://localhost:5106/",
    "RabbitMqUri": "amqp://curso:curso-local@localhost:5672/",
    "Prefijo": "aceria-m063",
    "WorkersActivos": false
  }
}
```

PASO 6.3.14 — Crear el archivo `ValidadorDecisionesTests.cs`

Las pruebas ejercitan aceptación y rechazos concretos del validador. Afirmar el diagnóstico evita un falso positivo por otro error.

1. Crea `tests/AceriaApp.Decisiones.Tests/ValidadorDecisionesTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.Decisiones.Tests/ValidadorDecisionesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Decisiones;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Decisiones.Tests;

public class ValidadorDecisionesTests
{
    private readonly ValidadorDecisiones _validador = new();

    [Fact]
    public void Validar_CatalogoPorDefecto_DevuelveValido()
    {
        var resultado = _validador.Validar(CatalogoDecisiones.DecisionesPorDefecto());

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinDecisiones_DevuelveError()
    {
        var resultado = _validador.Validar(new List<DecisionComunicacion>());
```

```
        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("No hay decisiones"));
    }

    [Fact]
    public void Validar_EnfoqueNoPermitido_DevuelveError()
    {
        var decisiones = new List<DecisionComunicacion>
        {
            new() { Interaccion = "X", Enfoque = "EnfoqueInventado" }
        };

        var resultado = _validador.Validar(decisiones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Enfoque no permitido"));
    }

    [Fact]
    public void Validar_SincronaSinRespuestaInmediata_DevuelveValido()
    {
        var decisiones = new List<DecisionComunicacion>
        {
            new() { Interaccion = "X", Enfoque = "Sincrona", RequiereRespuestaInmediata = false }
        };

        var resultado = _validador.Validar(decisiones);

        resultado.EsValida.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_AsincronaSinToleranciaEventual_DevuelveError()
    {
        var decisiones = new List<DecisionComunicacion>
        {
            new() { Interaccion = "X", Enfoque = "Asincrona", ToleraConsistenciaEventual = false }
        };

        var resultado = _validador.Validar(decisiones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no tolera consistencia eventual"));
    }

    [Fact]
    public void Validar_HibridaSinPatron_DevuelveError()
    {
        var decisiones = new List<DecisionComunicacion>
        {
            new() { Interaccion = "X", Enfoque = "Hibrida", TienePatronHibrido = false }
        };

        var resultado = _validador.Validar(decisiones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no declara patrón híbrido"));
    }
    [Fact]
    public void Validar_AsincronaConRespuestaInmediata_DevuelveError()
    {
        var decisiones = new List<DecisionComunicacion>
        {
            new() { Interaccion = "X", Enfoque = "Asincrona", RequiereRespuestaInmediata = true,
ToleraConsistenciaEventual = true }
        };

        var resultado = _validador.Validar(decisiones);

        resultado.EsValida.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("requiere respuesta inmediata del consumidor"));
    }
}
```

Crea la instancia del validador que utiliza cada prueba. Registra un caso ejecutable independiente en xUnit. Nombra la condición de prueba y su resultado esperado.

Ejecuta el validador sobre los datos preparados para este caso. Comprueba que la política acepta los datos de este caso. Comprueba que no se ha registrado ningún diagnóstico.

Comprueba que la política rechaza los datos de este caso. Comprueba el motivo concreto del rechazo; evita aceptar un fallo por otra regla. Prepara las entradas que se van a validar o devolver.

Verificación concreta. `ValidadorDecisionesTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

PASO 6.3.15 — Crear el archivo `CatalogoDecisionesTests.cs`

Las pruebas del catálogo verifican cantidades y propiedades declaradas. No prueban que existan los servicios nombrados.

1. Crea `tests/AceriaApp.Decisiones.Tests/CatalogoDecisionesTests.cs` dentro del proyecto correspondiente.
2. Copia el bloque completo siguiente respetando namespace, miembros y valores.
3. Guarda el archivo y ejecuta `dotnet build tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release`.
4. Comprueba los diagnósticos y corrige nombres o referencias antes de pasar al siguiente paso.

Archivo: `tests/AceriaApp.Decisiones.Tests/CatalogoDecisionesTests.cs`. **Operación:** Crear.

```
using AceriaApp.Api.Decisiones;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Decisiones.Tests;

public class CatalogoDecisionesTests
{
    [Fact]
    public void DecisionesPorDefecto_ContieneCincoDecisiones()
    {
        var decisiones = CatalogoDecisiones.DecisionesPorDefecto();

        decisiones.Should().HaveCount(5);
    }

    [Fact]
    public void DecisionesPorDefecto_ContieneDosSincronas()
    {
        var decisiones = CatalogoDecisiones.DecisionesPorDefecto();

        decisiones.Count(d => d.Enfoque == "Sincrona").Should().Be(2);
    }

    [Fact]
    public void DecisionesPorDefecto_ContieneDosAsincronas()
    {
        var decisiones = CatalogoDecisiones.DecisionesPorDefecto();

        decisiones.Count(d => d.Enfoque == "Asincrona").Should().Be(2);
    }

    [Fact]
    public void DecisionesPorDefecto_ContieneUnaHibrida()
    {
        var decisiones = CatalogoDecisiones.DecisionesPorDefecto();
```

```
        decisiones.Count(d => d.Enfoque == "Hibrida").Should().Be(1);
    }

    [Fact]
    public void DecisionesPorDefecto_HibridaTienePatron()
    {
        var decisiones = CatalogoDecisiones.DecisionesPorDefecto();

        decisiones.Where(d => d.Enfoque == "Hibrida").Should().OnlyContain(d => d.TienePatronHibrido);
    }
}
```

Obtiene los datos que se utilizarán en las comprobaciones siguientes. Comprueba el tamaño del catálogo sin inferir servicios o conexiones en ejecución. Compara el recuento obtenido con el valor esperado del catálogo.

Comprueba la condición en cada entrada declarada del catálogo.

Verificación concreta. `CatalogoDecisionesTests` compila en el proyecto indicado; su contenido coincide con el bloque completo.

Diagnóstico si falla.

- Un test sin `[Fact]` compila pero no se descubre: revisa los atributos del bloque.
- Añadir el archivo al proyecto anterior modifica su suite: crea el test en la ruta de este punto.

Implementación funcional: `AceriaApp.Decisiones.Tests.csproj`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj`. Sustituye el contenido de la copia actual.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><TargetFramework>net8.0</TargetFramework><Nullable>enable</Nullable><ImplicitUsings>enable</
ImplicitUsings><IsPackable>false</IsPackable><IsTestProject>true</IsTestProject></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="../../src/AceriaApp.Api/AceriaApp.Api.csproj" />
    <ProjectReference Include="../../src/AceriaApp.Inventario.Api/AceriaApp.Inventario.Api.csproj" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
    <PackageReference Include="xunit" Version="2.9.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
    <PackageReference Include="FluentAssertions" Version="6.12.2" />
    <PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="8.0.21" />
    <PackageReference Include="Testcontainers.MsSql" Version="3.10.0" />
</ItemGroup>
<ItemGroup><Compile Include="../Compartido/SqlServerPruebas.cs" Link="SqlServerPruebas.cs" /></ItemGroup>
</Project>
```

Declara una dependencia de compilación explícita; el ensamblado debe existir en la solución. Fija la versión de la dependencia para reproducir la compilación y sus API.

Implementación funcional: `FlujoHibridoTests.cs`

Este archivo completa el contrato, registro o prueba indicada por su nombre. Créalo en la ruta exacta para que la solución resuelva las dependencias del punto. Las responsabilidades de dominio, aplicación y adaptadores se mantienen separadas.

Archivo completo: `tests/AceriaApp.Decisiones.Tests/FlujoHibridoTests.cs`. Crea el archivo y sus carpetas si no existen.

```
using AceriaApp.Comunicacion.Aplicacion;
using AceriaApp.Comunicacion.Contratos;
using Xunit;
namespace AceriaApp.Decisiones.Tests;
public class FlujoHibridoTests
{
    private sealed class Planificador : IPlanificadorIntegrado
    {
        public IReadOnlyList<LineaPlanificadaV1>? Lineas = new[]{new LineaPlanificadaV1("Acero",20)};
        public int Escrituras;
        public Task<IReadOnlyList<LineaPlanificadaV1>?> ObtenerLineasAsync(Guid id,CancellationToken
ct)=>Task.FromResult(Lineas);
        public Task<EstadoPlanificacion> PlanificarAsync(Guid id,CancellationToken ct) { Escrituras++; return
Task.FromResult(EstadoPlanificacion.Aceptada); }
    }
    private sealed class Inventario(MaterialDto? valor,bool fallo=false) : IClienteInventario
    { public Task<MaterialDto?> ObtenerAsync(string material,CancellationToken ct)=>fallo?throw new
InventarioNoDisponibleException("No responde"):Task.FromResult(valor); }
    [Fact, Trait("Tipo","Unidad")]
    public async Task Disponibilidad_suficiente_delega_planificacion()
    {var p=new Planificador();Assert.Equal(EstadoPlanificacion.Aceptada,await new FlujoPlanificacion(p,new
Inventario(new("Acero",100))).EjecutarAsync(Guid.NewGuid(),default));Assert.Equal(1,p.Escrituras);}
    [Fact, Trait("Tipo","Unidad")]
    public async Task Stock_insuficiente_no_escribe_outbox()
    {var p=new Planificador();Assert.Equal(EstadoPlanificacion.SinDisponibilidad,await new
FlujoPlanificacion(p,new
Inventario(new("Acero",10))).EjecutarAsync(Guid.NewGuid(),default));Assert.Equal(0,p.Escrituras);}
    [Fact, Trait("Tipo","Unidad")]
    public async Task Material_ausente_no_escribe_outbox()
    {var p=new Planificador();Assert.Equal(EstadoPlanificacion.SinDisponibilidad,await new
FlujoPlanificacion(p,new
Inventario(null)).EjecutarAsync(Guid.NewGuid(),default));Assert.Equal(0,p.Escrituras);}
    [Fact, Trait("Tipo","Unidad")]
    public async Task Fallo_remoto_se_propaga_sin_escribir()
    {var p=new Planificador();await Assert.ThrowsAsync<InventarioNoDisponibleException>(()=>new
FlujoPlanificacion(p,new
Inventario(null,true)).EjecutarAsync(Guid.NewGuid(),default));Assert.Equal(0,p.Escrituras);}
    [Fact, Trait("Tipo","Unidad")]
    public async Task Orden_inexistente_no_consulta_servicio_remoto()
    {var p=new Planificador{Lineas=null};Assert.Equal(EstadoPlanificacion.NoExiste,await new
FlujoPlanificacion(p,new
Inventario(null,true)).EjecutarAsync(Guid.NewGuid(),default));Assert.Equal(0,p.Escrituras);}
    [Fact, Trait("Tipo","Unidad")]
    public async Task Agrupa_lineas_del_mismo_material_antes_de_comparar_stock()
    {var p=new Planificador{Lineas=new[]{new LineaPlanificadaV1("Acero",50),new
LineaPlanificadaV1("ACERO",50)}};Assert.Equal(EstadoPlanificacion.SinDisponibilidad,await new
FlujoPlanificacion(p,new
Inventario(new("Acero",80))).EjecutarAsync(Guid.NewGuid(),default));Assert.Equal(0,p.Escrituras);}
    [Fact, Trait("Tipo","Unidad")]
    public async Task Cancelacion_no_produce_planificacion()
    {var p=new Planificador();using var ct=new CancellationTokenSource();ct.Cancel();await
Assert.ThrowsAnyAsync<OperationCanceledException>(()=>new FlujoPlanificacion(p,new
Inventario(new("Acero",100))).EjecutarAsync(Guid.NewGuid(),ct.Token));Assert.Equal(0,p.Escrituras);}
}
```

Comprueba una consecuencia observable. Si cambia el comportamiento descrito por el nombre del test, esta aserción debe fallar.

PASO 6.3.16 — Compilar la solución completa

La comprobación final reúne compilación, pruebas de catálogo y ensayos de HTTP y mensajería real. Cada uno acredita una propiedad diferente y evita presentar una decisión declarada como un flujo ejecutado.

1. Aplica los bloques de integración funcional de este paso en el orden mostrado.
2. Añade a `AceriaApp.sln` todos los nuevos proyectos `.csproj` indicados por esos bloques.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Comprueba que la solución incluye 29 proyectos y termina sin errores.

Los archivos funcionales de este punto se han incorporado en los pasos 5, 8, 9, 10, 13 y 15. Antes de compilar, agrega todos los proyectos nuevos a la solución con los comandos indicados y guarda los archivos completos.

Verificación concreta. La solución de la etapa compila con 29 proyectos.

Diagnóstico si falla.

- Omitir un proyecto de la solución impide verificarlo con `dotnet test AceriaApp.sln`: compara las rutas `.csproj` y el listado de la solución.
- Interpretar un test de catálogo como prueba de tráfico omite fallos de transporte: ejecuta también la integración funcional.

PASO 6.3.17 — Ejecutar todos los tests del punto 6.3

La prueba del punto distingue reglas de análisis y comportamiento de sus adaptadores.

1. Ejecuta `dotnet test tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release --no-build --no-restore`.
2. Comprueba que se ejecutan los 12 tests de catálogo y las pruebas funcionales añadidas al proyecto hasta el paso 15.
3. Revisa que cada prueba figure como aprobada, sin errores ni omisiones.
4. Conserva el resultado de ejecución para compararlo con la suite completa del paso siguiente.

Verificación concreta. Los 12 casos de catálogo se descubren y pasan junto con la integración del punto.

Diagnóstico si falla.

- Usar `--no-build` después de editar ejecuta binarios anteriores: vuelve a compilar antes de probar.
- Un puerto ocupado o una dependencia no disponible exige corregir el entorno; no conviertas las pruebas en omitidas.

PASO 6.3.18 — Ejecutar todos los tests de la solución

La suite completa detecta regresiones de dominio, persistencia y comunicación. Cada proyecto produce su propio resumen; una sola línea de salida no es el total de la solución.

1. Configura `ACERIA_TEST_SQLSERVER` para SQL Server Express o LocalDB y comprueba la disponibilidad de los servicios de integración.
2. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore --logger trx --results-directory TestResults`.
3. Suma los resultados de los archivos TRX y verifica 283 pruebas aprobadas, cero fallos y cero omisiones.
4. Revisa las pruebas heredadas de M03, M04 y M05: deben seguir formando parte de la solución.

Verificación concreta. Los TRX suman 283 pruebas aprobadas sin omitir integración con SQL Server.

Diagnóstico si falla.

- Un error de conexión SQL Server no autoriza sustituirlo por memoria: comprueba instancia, credenciales y permiso para crear bases de prueba.
- Comparar solo el último resumen de `dotnet test` infracuenta la solución: suma todos los TRX.

Comprobación con procesos reales

Utiliza las dos terminales y las direcciones del apartado «Arranque de los dos procesos». Comprueba que Inventario escucha en 5106 y Producción en 5105. Los tests de cliente con un handler controlado y los de WebApplicationFactory no reemplazan este recorrido por sockets.

Crea una orden desde otra terminal; no uses el endpoint de planificación anterior de M5 para este recorrido:

```
$orden=Invoke-RestMethod -Method Post -Uri http://localhost:5105/api/v1/produccion/ordenes -ContentType
'application/json' -Body '{"cliente":"Axia","material":"Acero","cantidad":20}'
$id=$orden.id
Invoke-RestMethod -Method Post -Uri "http://localhost:5105/api/v1/comunicacion/ordenes/$id/planificar"
Invoke-RestMethod -Uri "http://localhost:5105/api/v1/produccion/ordenes/$id"
Invoke-RestMethod -Uri "http://localhost:5106/api/v1/planificaciones/$id"
```

La planificación devuelve 202. La orden queda `Planificada`; la última consulta puede devolver 404 mientras el evento siga pendiente y 200 tras procesarlo. Repite esa consulta durante unos segundos, con un límite de espera explícito; una demora no es garantía de entrega futura. Con los workers activos, la proyección muestra `cantidadTotal=20`. Detén y vuelve a arrancar Inventario: esa proyección debe seguir en SQL. Publicador y consumidor reintentan los fallos técnicos con pausa; revisa sus logs si el estado no avanza. Los mensajes incompatibles van a la cola terminada en `.errores`; un operador debe examinarlos antes de un reproceso deliberado.

La llamada a `/api/v1/produccion/ordenes/{id}/planificar` sigue siendo el ejercicio local heredado de M5 y no genera eventos. El recorrido integrado usa siempre `/api/v1/comunicacion/ordenes/{id}/planificar`. Mantener ambas rutas permite comparar sus contratos sin atribuir mensajería a una operación que no la implementa.

En 6.3, una orden de 450 unidades de Acero obtiene 409 de disponibilidad y permanece en borrador porque el catálogo real de Inventario contiene 300. Una caída de Inventario obtiene 503 y no escribe outbox. Dos líneas del mismo material se suman antes de comparar. La comprobación no reserva ni descuenta stock: el estado Planificada expresa la regla local del laboratorio.

PASO 6.3.19 — Actualizar el README del punto 6.3

El README de la copia debe describir el estado actual y la evidencia disponible, no acumular instrucciones obsoletas.

1. Abre el `README.md` copiado dentro de `6.3-cuando-usar-cada-enfoque`.
2. Reemplaza su contenido completo por el bloque siguiente.
3. Comprueba sus enlaces relativos y las instrucciones de arranque y verificación.
4. Guarda el archivo y confirma que el README de `6.2-comunicacion-asincrona-eventos` no se ha modificado.

Archivo: `README.md`. **Operación:** Reemplazar el contenido completo.

```
# Punto 6.3 — Cuándo utilizar cada enfoque

## Objetivo

Completar el catálogo y su validación, distinguirlo de los adaptadores ejecutables y conservar la base
anterior.

## Estado del proyecto

- Solución `AceriaApp.sln`: 29 proyectos y 283 pruebas acumuladas.
- 12 pruebas nuevas de análisis en `tests/AceriaApp.Decisiones.Tests`.
- Tipos `DecisionComunicacion`, `ValidadorDecisiones` y `CatalogoDecisiones` en `src/AceriaApp.Api/
Decisiones`.
- Los catálogos no demuestran tráfico HTTP ni entrega de mensajes.
```

```
## Cómo ejecutarlo

1. Abre `AceriaApp.sln` en Visual Studio con soporte .NET 8.
2. Configura SQL Server Express o LocalDB y la conexión de pruebas `ACERIA_TEST_SQLSERVER`; conserva el
helper `tests/Compartido/SqlServerPruebas.cs`.
3. Ejecuta `dotnet restore AceriaApp.sln` y `dotnet build AceriaApp.sln -c Release`.
4. Ejecuta `dotnet test AceriaApp.sln -c Release --no-build --no-restore` con los servicios requeridos por la
integración funcional disponibles.
5. Sigue el manual M06 para iniciar los adaptadores y observar cada recorrido de comunicación.

## Qué se ha añadido respecto al punto anterior

El catálogo del punto, su proyecto de pruebas, `docs/cuando-usar-cada-enfoque.md` y el ADR-026. Los bloques
funcionales de los pasos 5 a 15 completan la integración del punto. Se conservan los documentos y pruebas
heredados.

## Documentación

- [Tema del punto](docs/cuando-usar-cada-enfoque.md).
- [ADR-026](docs/ADR-026-cuando-usar-cada-enfoque.md).
- Los 32 documentos anteriores, incluidos los ADR-010 a ADR-025, permanecen disponibles en `docs`.
```

Verificación concreta. El README identifica 29 proyectos, 283 pruebas y 12 tests nuevos de catálogo.

Diagnóstico si falla.

- Crear otro README junto al heredado deja ambiguo cuál seguir: reemplaza el de la raíz de la etapa.
- Enlazar ADR que no existen confunde el recorrido: utiliza los enlaces del bloque.

PASO 6.3.20 — Crear el ADR-026 del punto 6.3

El ADR registra la decisión, alternativas y consecuencias sin reescribir decisiones anteriores.

1. Crea `docs/ADR-026-cuando-usar-cada-enfoque.md` dentro de la copia de `6.3-cuando-usar-cada-enfoque`.
2. Introduce el contenido completo del bloque siguiente.
3. Guarda el archivo en UTF-8 y revisa su vista previa Markdown.
4. Comprueba que `docs/ADR-026-cuando-usar-cada-enfoque.md` existe y que los 32 documentos anteriores permanecen intactos.

Archivo: `docs/ADR-026-cuando-usar-cada-enfoque.md`. **Operación:** Crear.

```
# ADR-026: Cuándo utilizar cada enfoque

## Estado

Aceptado.

## Contexto

AceriaApp continúa la etapa anterior y necesita razonar explícitamente sobre sus interacciones. La solución
conserva los modelos y pruebas heredados; el catálogo nuevo es un ejercicio de análisis separado de los
adaptadores de comunicación ejecutables.

## Decisión

Elegir por interacción considerando respuesta requerida, consistencia, fallos y operación. Admitir síncrona
aunque no sea obligatoria una respuesta inmediata, evitar prometer resultado inmediato en una notificación
asíncrona y describir el protocolo de las interacciones híbridas.

## Alternativas consideradas

- Elegir un solo estilo para todo el sistema: descartado porque las interacciones tienen necesidades
diferentes.
- Omitir los criterios de recuperación y consistencia: descartado porque deja ambiguos los fallos parciales.
- Considerar suficiente la validación del catálogo: descartado porque no comprueba la comunicación
ejecutable.

## Consecuencias
```

```
- Positivas: decisiones explícitas, casos de rechazo reproducibles y trazabilidad por interacción.
- Negativas: mantener contratos, políticas y pruebas de comportamiento añade trabajo.
- Mitigación: distinguir pruebas de análisis y pruebas funcionales, mantener límites claros y revisar las
decisiones con evidencia operativa.
```

Verificación concreta. `docs/ADR-026-cuando-usar-cada-enfoque.md` incluye todos los apartados del bloque mostrado.

Diagnóstico si falla.

- Guardar el archivo fuera de `docs` rompe los enlaces del README: usa `docs/ADR-026-cuando-usar-cada-enfoque.md`.
- Copiar un recuento histórico sin comprobarlo confunde el estado del punto: utiliza los números de esta etapa.

PASO 6.3.21 — Verificar que no se ha modificado ningún archivo del punto 6.2

Los archivos nuevos se añaden en la copia actual; los documentos y tests heredados siguen siendo la base de regresión.

1. Abre PowerShell en la raíz del curso y define las carpetas anterior `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.2-comunicacion-asincrona-eventos` y actual `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque`.
2. Enumera los archivos heredados de `docs` y de `tests` terminados en `.cs` en la etapa anterior.
3. Compara cada archivo con su copia actual mediante `Get-FileHash`; el mismo camino relativo debe tener el mismo SHA-256.
4. Comprueba 32 documentos idénticos y restaura únicamente cualquier documento o prueba heredada que se haya alterado por error.

Verificación concreta. Los 32 documentos y todas las pruebas heredadas tienen el mismo SHA-256; los archivos nuevos del punto no se comparan como heredados.

Diagnóstico si falla.

- Comparar solo fechas no detecta cambios de contenido: utiliza hashes.
- Usar la lista de archivos de la etapa actual incluye los nuevos y produce falsos ausentes: enumera desde la etapa anterior.

PASO 6.3.22 — Verificar la estructura del punto 6.3

La estructura debe permitir reconstruir y ejecutar el punto desde sus archivos y referencias, sin proyectos ocultos fuera de la etapa.

1. Ejecuta `tree /F` en Windows desde la carpeta de la etapa.
2. Localiza las tres clases de `src/AceriaApp.Api/Decisiones` y las dos clases de tests en `tests/AceriaApp.Decisiones.Tests`.
3. Comprueba los 34 documentos de `docs`, incluida la ficha del punto y el ADR-026.
4. Ejecuta `dotnet sln AceriaApp.sln list` y contrasta sus 29 proyectos con los `.csproj` presentes.

Verificación concreta. Las rutas del catálogo, documentación e integración están presentes y la solución incluye 29 proyectos.

Diagnóstico si falla.

- Un proyecto enlazado fuera de la etapa puede compilar solo en tu equipo: comprueba que todas las rutas sean relativas y estén dentro de la copia.
- Omitir el segundo archivo de tests pierde las comprobaciones del catálogo: comprueba ambos nombres.

PASO 6.3.23 — Verificar el `.gitignore` del punto 6.3

Las exclusiones se aplican desde la raíz a las etapas; no es necesario inventar un `.gitignore` propio dentro de cada copia.

1. Abre el `.gitignore` de la raíz del repositorio del curso.
2. Comprueba las reglas `**/bin/`, `**/obj/` y `**/TestResults/` que ya excluyen los artefactos de todas las etapas.
3. Ejecuta `git status --short` y revisa que no aparezcan binarios, cachés ni resultados temporales.
4. Conserva las exclusiones existentes; si aparecen archivos locales de Visual Studio, excluye `.vs/`, `*.user` y `*.suo` mediante `.git/info/exclude`, sin ignorar código ni configuración entregable.

Verificación concreta. Los archivos compilados y resultados de tests están excluidos; los archivos del punto siguen visibles para Git.

Diagnóstico si falla.

- Ignorar todo `tests/` elimina la evidencia reproducible: excluye solo sus artefactos generados.
- Esperar un `.gitignore` dentro de cada etapa puede llevar a crear duplicados innecesarios: revisa primero el de la raíz.

PASO 6.3.24 — Hacer el commit del punto 6.3 y verificar el estado final del módulo 6

Un commit local identifica el estado comprobado del punto; su número depende de la historia real y no se deduce del temario.

1. Ejecuta `git diff --check` y revisa `git status --short` desde la raíz del curso.
2. Añade los archivos revisados de `M06_COMUNICACION_MICROSERVICIOS/PROYECTO/6.3-cuando-usar-cada-enfoque` con `git add` y comprueba el resumen de `git diff --cached --stat`.
3. Cuando las pruebas hayan pasado, crea el commit local con el mensaje `Punto 6.3: cuándo utilizar cada enfoque`.
4. Comprueba `git log -1 --oneline` y `git status --short`; no incluyas cambios ajenos al punto.

Verificación concreta. El commit local contiene solo los cambios revisados de 6.3 y los archivos esperados quedan registrados.

Diagnóstico si falla.

- Hacer commit antes de revisar el índice puede incorporar trabajo ajeno: inspecciona los archivos staged.
- Afirmar un número de commits igual al número de lecciones no verifica la historia real: consulta `git log`.

Reto de ampliación (optativo)

Añade `Resumen` a `ValidadorDecisiones` y una prueba adicional. Trabaja en una copia optativa de `6.3-cuando-usar-cada-enfoque` fuera de `PROYECTO`; el reto no se hereda en el punto siguiente.

Reto 6.3.1 — Definir el contrato del resumen

Abre `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs` e inserta el contrato después del `record` de resultado, en el mismo namespace.

Archivo: `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs`. **Operación:** Insertar después del ancla indicada.

Ancla exacta: `public record ResultadoDecisiones(bool EsValida, IReadOnlyList<string> Errores);`. Debe aparecer una sola vez.

```
public record ResumenDecisiones(int Total, int Sincronas, int Asincronas, int Hibridas);
```

Reto 6.3.3 — Calcular los recuentos

Inserta el método dentro de `ValidadorDecisiones`, antes del campo `private static readonly string[];` conserva las cuatro posiciones del constructor del resumen.

Archivo: `src/AceriaApp.Api/Decisiones/ValidadorDecisiones.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `private static readonly string[]`. Debe aparecer una sola vez.

```
    public static ResumenDecisiones Resumen(IEnumerable<DecisionComunicacion> decisiones)
    {
        var lista = decisiones.ToList();
        var sincronas = lista.Count(d => d.Enfoque == "Sincrona");
        var asincronas = lista.Count(d => d.Enfoque == "Asincrona");
        var hibridas = lista.Count(d => d.Enfoque == "Hibrida");
        return new ResumenDecisiones(lista.Count, sincronas, asincronas, hibridas);
    }
```

Cuenta las entradas que cumplen el criterio indicado. Devuelve los recuentos del catálogo sin modificar sus entradas.

Reto 6.3.6 — Verificar el catálogo por defecto

Inserta la prueba dentro de `ValidadorDecisionesTests`, antes del campo `_validador`. La prueba comprueba total 5, síncronas 2, asíncronas 2 e híbridas 1.

Archivo: `tests/AceriaApp.Decisiones.Tests/ValidadorDecisionesTests.cs`. **Operación:** Insertar antes del ancla indicada.

Ancla exacta: `private readonly ValidadorDecisiones _validador = new();`. Debe aparecer una sola vez.

```
    [Fact]
    public void Resumen_ConCatalogoPorDefecto_CalculaTotales()
    {
        var resumen = ValidadorDecisiones.Resumen(CatalogoDecisiones.DecisionesPorDefecto());

        resumen.Total.Should().Be(5);
        resumen.Sincronas.Should().Be(2);
        resumen.Asincronas.Should().Be(2);
        resumen.Hibridas.Should().Be(1);
    }
```

Calcula el resumen sobre el catálogo por defecto.

Comprobación del reto. Compila la copia optativa y ejecuta primero `dotnet test tests/AceriaApp.Decisiones.Tests/AceriaApp.Decisiones.Tests.csproj -c Release`, después `dotnet test AceriaApp.sln -c Release`. El catálogo pasa de 12 a 13 tests; la solución debe ejecutar 283 + 1 pruebas, todas aprobadas y ninguna omitida. Confirma en el TRX que se ejecutó `Resumen_ConCatalogoPorDefecto_CalculaTotales` y que la etapa obligatoria conserva sus hashes.

Diagnóstico si falla. Si el método queda fuera de la clase, mueve solo el fragmento al ancla señalada. Si el total no coincide, revisa las entradas del catálogo antes de cambiar la expectativa. No acumules los retos al construir la siguiente etapa.

Resultado esperado del punto

La etapa contiene las tres clases del análisis, sus 12 pruebas de catálogo, los adaptadores funcionales y 34 documentos Markdown. La solución incluye 29 proyectos y 283 pruebas aprobadas. Los documentos y pruebas heredados permanecen idénticos.